**ALEXANDRIA UNIVERSITY**
**FACULTY OF ENGINEERING**
**Computer and Communication Engineering**

# Adversarial Attacks on Deep Learning Models

A Thesis submitted in fulfillment of the requirements for
The Degree of B.Sc in Computer and Communication Engineering

by

**Menna Ibrahim** **Adham Wael**
**Amr Mohamad** **Nour El-Din Hazem**

Department of Computer and Communication
Faculty of Engineering, Alexandria University.

Supervised By
**Dr. Marwan Torki**
**Dr. Ahmed Kosba**

Alexandria, 2023

# Acknowledgment

First and foremost, we express our profound gratitude to Allah for providing us with the strength and guidance to persevere through the challenges of this year and for blessing our efforts with success. We would also like to extend our sincerest appreciation to our esteemed supervisors, Dr. Marawan Torki and Dr. Ahmed Kosba, for their unwavering support and mentorship throughout the course of this thesis.

Their invaluable expertise, constructive feedback, and dedication to our academic growth have been instrumental in shaping our work and ensuring its quality. We have gained a wealth of knowledge under their tutoring, and we look forward to the possibility of further collaborations in pursuit of greater achievements in the future.

# Abstract

Adversarial Machine Learning plays a critical role in the domain of Natural Language Processing (NLP), where its primary objective is to identify and analyze vulnerabilities in Machine Learning (ML) models, subsequently exploiting them with precision. Adversarial examples are carefully crafted to deceive ML models while appearing innocuous to human observers.

The aim of adversarial ML is twofold: to assess model vulnerabilities by generating adversarial examples and to foster the development of more robust algorithms and models. By detecting adversarial examples, it contributes to enhancing model robustness.

This study focuses on two distinct types of adversarial attacks: Char-level Attacks and Word-level Attacks. Char-level Attacks manipulate the fine-grained aspects of individual characters in the text, encompassing techniques such as inner shuffle attacks, flip attacks, special characters attacks, space attacks, and more. On the other hand, Word-level Attacks target individual words or groups of words, involving strategies like the Bert Attack.

Our research endeavors have been directed towards the Arabic language, driven by its underexplored status in the realm of adversarial machine learning. This decision stems from the realization that the Arabic language presents substantial disparities, encompassing variations in alphabet, structure, and morphology when compared to English. These distinctions make the investigation of adversarial machine learning within the Arabic language a particularly compelling and vital pursuit.

# Contents

**Acknowledgment** **iii**

**Abstract** **v**

**Table of Contents** **7**

**List of Figures** **9**

**List of Tables** **11**

**List of Abbreviations** **13**

**1 Introduction** **1**
1.1 Motivation . . . 1
1.2 Problem Statement . . . 3
1.3 Structure Overview . . . 3

**2 Background and Related Work** **5**
2.1 Background . . . 5
2.1.1 Natural Language Processing . . . 5
2.1.2 Adversarial Attacks on Deep Learning . . . 20
2.2 Related Work . . . 30
2.2.1 Attacks on Other Languages . . . 30
2.2.2 Attacks on Arabic Language . . . 32

**3 Methodology** **35**
3.1 Introduction . . . 35
3.2 Naïve Approach for Arabic specific Attacks . . . 36
3.2.1 Adjective Attack . . . 36
3.2.2 Flip Attack . . . 38
3.3 Scoring Function . . . 40
3.4 Introduction to our Approach . . . 43
3.5 Candidate Generation . . . 44
3.5.1 Word-Level Attacks . . . 44
3.5.2 Character-Level Attacks . . . 48
3.6 Search Algorithms . . . 52

3.6.1 Greedy Algorithm . . . . . 53
3.6.2 Particle Swarm Optimization . . . . . 54

**4 Experimental Setup and Implementation Details 61**
4.1 Dataset . . . . . 61
4.1.1 Overview . . . . . 61
4.1.2 Label Statistics and Data Partitioning . . . . . 62
4.1.3 Data Cleaning . . . . . 63
4.2 Models . . . . . 63
4.2.1 Parameters . . . . . 64
4.2.2 Training Process Results . . . . . 64
4.2.3 Plots . . . . . 65

**5 Experiments and Results 69**
5.1 Naive Approach vs. Our Approach . . . . . 69
5.2 Hyperparameters of MarBERT and AraBERT attacks . . . . . 71
5.3 Study on AraBERT and MarBERT finetuned models using Offensive Data . . . . . 72
5.4 Study on AraBERT model using Offensive and Non-offensive Data . 78

**6 Results Analysis 81**
6.1 Relationship between Attack Performance, Perceptibility, and Validity 81
6.2 Transformer-based models: AraBERT and MarBERT . . . . . 84
6.3 Effects of Attacks on Offensive and Non-Offensive Sentences . . . . 86
6.4 Advantages of our approach . . . . . 87

**7 Future Work 89**
7.1 Explore Different Search Algorithms . . . . . 89
7.2 Explore Alternative Candidate Generation . . . . . 90
7.2.1 Sememes . . . . . 90
7.3 Adversarial Defences . . . . . 92
7.3.1 Adversarial Purification . . . . . 92
7.3.2 Fuzzy Matching . . . . . 93
7.4 Adversarial Attacks . . . . . 95

**8 Conclusion 97**

**References 99**

# List of Figures


2.1 Illustration of self-attention calculation in transformer . . . . . 13

3.1 Illustration of BERT attack replacement process . . . . . 46

4.1 Accuracy-Epoch Plot for AraBERT Model Training. . . . . 66

4.2 Loss-Epoch Plot for AraBERT Model Training. . . . . 66

4.3 Accuracy-Epoch Plot for MarBERT Model Training. . . . . 67

4.4 Loss-Epoch Plot for MarBERT Model Training. . . . . 67

5.1 Prediction Scores Graph . . . . . 72

5.2 Plots of the accuracy change with the change limits in MarBERT, AraBERT, Flip, English Flip, Middle Space, and One Space Anywhere Attacks on AraBERT model . . . . . 74

5.3 Plots of the accuracy change with the change limits in Middle Special Character, One Special Character Anywhere, Separate Letters by Spaces, Separate Letters by Special Characters, Inner Shuffle,and Adjective attacks on AraBERT model . . . . . 75

5.4 Plots of the accuracy change with the change limits in MarBERT, AraBERT, Flip, English Flip, Middle Space, and One Space Anywhere Attacks on MarBERT model . . . . . 76

5.5 Plots of the accuracy change with the change limits in Middle Special Character, One Special Character Anywhere, Separate Letters by Spaces, Separate Letters by Special Characters, Inner Shuffle,and Adjective attacks on MarBERT model . . . . . 77

# List of Tables


3.1 Example on the Adjective Attack transformations for the word 'جيد' 38

3.2 Adjective Attack Example . . . . . 38

3.3 Resemblance groups in Arabic Alphabet . . . . . 39

3.4 Types of Resemblance groups in Arabic Alphabet . . . . . 39

3.5 Scoring different examples using our scoring function with class 3 as the correct class . . . . . 43

3.6 BERT Attack Example . . . . . 48

3.7 Arabic Transliterations . . . . . 51

3.8 Special Cases in Arabic Transliterations . . . . . 51

3.9 Character-level Attacks Example . . . . . 52

4.1 Representation of the data split and number of each label in the OSACT5 Dataset. . . . . 63

4.2 Parameters used for training AraBERT and MarBERT Models. . . 64

4.3 AraBERT and MarBERT training process results. . . . . 65

5.1 Comparison between basic Adjective Attacks (Definite, Dual, Plural and Gender) and our modified approach at different change limits 0.4 and 1 on AraBERT using previously correctly classified offensive sentences (model's accuracy is 100% prior to the attack) . . . . . 70

5.2 Comparison between Ar-Flip1 and Ar-Flip2 attacks using the naive approach and Flip Attack using our approach at different change limits 0.4 and 1 on AraBERT using previously correctly classified offensive sentences (model's accuracy is 100% prior to the attack) . 71

5.3 Hyperparameters used in AraBERT and MarBERT Attacks . . . . 72

5.4 Comparison between AraBERT & MarBERT Models accuracies after each attack on previously correctly classified offensive sentences (starting from 100% accuracy) with change limit = 1 . . . . . 73

5.5 AraBERT Model accuracy after each attack on previously correctly classified sentences (starting from 100% accuracy) with change limit = 0.4 . . . . . 78

5.6 Average Query Numbers on AraBERT Model . . . . . 79

5.7 Average Perturbation Percentage on AraBERT Model . . . . . 79

5.8 Average word length of the Misclassified, Failed, Misclassified Offensive, Misclassified Non-offensive sentences in the dataset in AraBERT experiment. The average length of sentence in the entire data is 12 words. . . . . 80

# List of Abbreviations

| | |
|---|---|
| **ML** | **M**achine **L**earning |
| **LM** | **L**anguage **M**odel |
| **NLP** | **N**atural **L**anguage **P**rocessing |
| **CNN** | **C**onvolutional **N**eural **N**etwork |
| **RNN** | **R**ecurrent **N**eural **N**etwork |
| **LSTM** | **L**ong **S**hort-**T**erm **M**emory |
| **GRU** | **G**ated **R**ecurrent **U**nit |
| **BERT** | **B**idirectional **E**ncoder **R**epresentations from **T**ransformers |
| **RoBERTa** | **R**obustly **O**ptimized **BERT** Pre-training **A**pproach |
| **GPT** | **G**enerative **P**re-trained **T**ransformer |
| **NSP** | **N**ext **S**entence **P**rediction |
| **MSA** | **M**odern **S**tandard **A**rabic |
| **BPE** | **B**yte **P**air **E**ncoding |
| **HMM** | **H**idden **M**arkov **M**odel |
| **PCFG** | **P**robabilistic **C**ontext-**F**ree **G**rammar |
| **OSACT5** | **O**pen **S**ource **A**rabic **C**orbora and Processing **T**ools **5**th edition |
| **OSIAN** | **O**pen **S**ource **I**nternational **A**rabic **N**ews corpus |
| **GLUE** | **G**eneral **L**anguage **U**nderstanding **E**valuation |
| **OOV** | **O**ut **O**f **V**ocabulary |
| **OFF** | **Off**ensive |
| **NOT_OFF** | **Not Off**ensive |
| **R1S** | **R**eplace-**1** **S**coring function |
| **MLM** | **M**asked **L**anguage **M**odel |
| **POS** | **P**art **O**f **S**peech |
| **PSO** | **P**article **S**warm **O**ptimization |

# Chapter 1

# Introduction

## 1.1 Motivation

The rapid growth of social media platforms and online communication has brought about numerous benefits in terms of connectivity and information sharing. However, it has also given rise to a concerning issue: the increase of hate speech and offensive content. With the anonymity that the internet grants its users, there exists a darker aspect characterized by heightened cybercrimes facilitated by concealed identities. The shield of anonymity often emboldens individuals to express harsh and offensive language, taking advantage of the perceived lack of accountability.

The impact of hate speech can be far-reaching, leading to social tensions, discrimination, and even violence. Consequently, the need for effective hate speech detection mechanisms has become increasingly critical. By acknowledging the challenges posed by anonymous online activities and embracing a proactive approach to content control, we can foster a healthier and more secure digital ecosystem.

The exponential growth of online data has rendered human surveillance of this vast volume virtually unattainable, necessitating the adoption of automated tools to alleviate the reliance on manual labor. To achieve this automation, the development of a tool capable of comprehending human language and discerning patterns within the data becomes imperative, bringing us back to the realm of Natural Language Processing (NLP).

While extensive research has been conducted in NLP in various languages, including English, the problem remains largely underexplored in the context of Arabic. Arabic is one of the most widely spoken languages globally, with a vast online presence and a diverse user base. Despite the prevalence of Arabic content, the development of robust models for this language is still limited.

Moreover, another significant challenge lies in the adversarial attacks employed by internet users to bypass existing hate speech detection models. Adversarial attacks involve malicious users intentionally manipulating their content to evade detection, often employing subtle linguistic tricks or altering the context of their messages. These attacks pose a substantial obstacle to the efficacy of hate speech detection systems, as they exploit vulnerabilities and render the models ineffective.

Addressing these challenges requires a comprehensive understanding of hate speech in Arabic, the linguistic nuances specific to the language, and the complex nature of adversarial attacks in the context of natural language processing (NLP).

By delving into the depths of adversarial attacks, this project endeavors to simulate the seriousness of these attacks and pave the way for future research in devising robust defenses against them. Moreover, it aims to shed light on the vulnerabilities inherent in machine learning models, serving as a call to raise awareness regarding these weaknesses before their blind adoption.

Furthermore, the outcomes of this research will not only benefit the academic community but also possibly open the door for more research in finding practical solutions for social media platforms. Given the underexplored nature of NLP in Arabic and the persistent challenges posed by adversarial attacks, this research strives to advance the field of NLP in Arabic, shed light on the serious impact of adversarial examples to help to cultivate safer online environments, and safeguard individuals, and communities from the heinous consequences of hate speech.

## 1.2 Problem Statement

The increasing reliance on machine learning models, particularly transformer-based models, in various domains has led to a growing concern about their vulnerability to adversarial attacks. Adversarial attacks refer to the deliberate manipulation of input data to deceive or mislead machine learning models, compromising their performance and reliability. Despite the advancements in the field, the effectiveness and impact of character-level and word-level adversarial attacks on transformer-based models, such as AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020), remain insufficiently understood. This lack of comprehensive understanding hinders the development of robust defenses and effective countermeasures against such attacks. Therefore, the problem addressed in this thesis is to investigate and analyze the vulnerability of these models to character-level and word-level adversarial attacks, assess their impact on model performance and robustness, and explore potential strategies to enhance the resilience of these models against adversarial manipulations.

## 1.3 Structure Overview

This section illustrates the structure of our thesis. It could be described as follows: ***Chapter 1*** Motivates our problem and why it lured our attention. It then clearly states the problem and finally structures the thesis. ***Chapter 2*** Chronologize the background behind NLP and its advancements. Afterward, it explains the essence of adversarial attacks and discusses all their aspects and various factors. Then lastly, it provides the previous work related to our conducted study. ***Chapter 3*** Elaborates on the methodology behind our work, depicts our contribution, and describes the prospects of our study. ***Chapter 4*** Sets up our experiment and demonstrates the details of our implementations. ***Chapter 5*** Frames the experiments performed and reports the results and visualizations. ***Chapter 6*** Discusses our analysis of the conducted results and the observations and conclusions this study establishes. ***Chapter 7*** Proposes future research that could open the door for impactful contributions. ***Chapter 8*** Concludes our thesis and the study performed, and our assertions.

# Chapter 2

# Background and Related Work

## 2.1 Background

### 2.1.1 Natural Language Processing

NLP, which stands for Natural Language Processing, is a field of artificial intelligence and computational linguistics that focuses on the interaction between computers and human language. It involves the study and development of algorithms and models that enable computers to understand, interpret, and generate human language in a way that is meaningful and useful.

NLP is a multidisciplinary field. At its core, NLP aims to bridge the gap between human language and machine understanding. It involves a wide range of techniques and approaches, including statistical modeling, machine learning, deep learning, linguistics, and cognitive science.

One of the fundamental challenges in NLP is the inherent complexity and ambiguity of natural language. Human language, in contrast to structured data, displays a great deal of variation, including various sentence forms, grammar conventions, idioms, and contextual complexity. This variability poses significant challenges when it comes to designing algorithms and models that can process and comprehend language accurately.

To analyse and comprehend language, NLP mainly relies on machine learning techniques and models. To create predictions and carry out various language-related

activities, these algorithms extract patterns and relationships from vast amounts of annotated data. Supervised learning, unsupervised learning, and reinforcement learning techniques are commonly employed in NLP.

Supervised learning involves training models on labeled datasets, where the inputs are paired with their corresponding desired outputs. This approach is used for tasks like hate speech detection, sentiment analysis, named entity recognition, and machine translation.

Unsupervised learning, on the other hand, involves training models on unlabeled data to discover patterns and structures. This is useful for tasks like topic modeling, clustering, and word embeddings.

Reinforcement learning is employed to develop systems that can interact with their environment and learn through trial and error. Reinforcement learning has been applied to dialogue systems and chatbots.

Deep learning, a subfield of machine learning, has revolutionized NLP in recent years. Deep learning models, such as recurrent neural networks (RNNs), convolutional neural networks (CNNs), and transformers, have achieved state-of-the-art performance on a wide range of NLP tasks. These models can capture complex patterns and dependencies in language data, enabling more accurate and nuanced language understanding.

In NLP, ethical issues are equally crucial. When utilising NLP technologies and working with massive language datasets, bias, fairness, and privacy concerns surface. In order to ensure that NLP technologies are used ethically and inclusively, researchers and practitioners in the field are drafting ethical guidelines.

#### 2.1.1.1 NLP Tasks

Due to its capacity to analyze, comprehend, and produce human language, NLP has drawn considerable attention and has been utilized across numerous areas. NLP covers a broad range of activities, including as generation and classification, each with a wealth of real-world uses.

In the generation domain, NLP enables the automatic production of human-like language output based on given input or data. As a result, developments have

been made in fields like machine translation, data-driven storytelling, personalized email responses, and automatic report generation. By drawing conclusions from structured data or analytics, for instance, NLP-powered systems can provide in-depth reports, giving firms useful information in a clear and intelligible way. The development of customized email responses or product recommendations has also been made easier by the use of natural language generation tools, increasing consumer engagement and satisfaction.

On the other hand, NLP plays a pivotal role in classification tasks, where the objective is to assign predefined categories or labels to texts based on their content. Numerous domains can benefit from the use of classification tasks, which range from subject categorization and sentiment analysis to more complicated applications like social meaning and hate speech identification. Understanding the underlying social context, connotations, and cultural consequences of textual content is the emphasis of social meaning classification. NLP systems can correctly analyze and react to text by determining social meanings, taking into account things like sarcasm, irony, or informal language usage.

Hate speech identification stands out as a crucial task in the classification field. Hate speech refers to language or expressions that propagate violence, discrimination, or hostility towards individuals or groups based on attributes such as race, ethnicity, religion, gender, or other protected characteristics. In order to address the harmful effects of toxic online discourse and promote a safer digital environment, hate speech identification has become increasingly crucial. By leveraging NLP techniques, hate speech detection models can automatically identify and categorize instances of hate speech in text data, contributing to the identification and mitigation of harmful content.

In order to accurately identify hate speech, it is necessary to analyze language characteristics and contextual cues. Various NLP methods and algorithms are employed for this task, such as supervised machine learning algorithms, lexicon-based approaches, and deep learning models. These approaches leverage annotated datasets, feature engineering, and large-scale language models to detect hate speech accurately. Ethical considerations, including fairness and bias mitigation, are of utmost importance in hate speech detection to ensure that models do not perpetuate discrimination or unintended biases.

#### 2.1.1.2 Hate Speech Detection Task

Detecting hate speech in Arabic poses unique challenges due to the linguistic diversity encompassing various dialects and spelling variations. Traditional approaches, including the use of models trained on Modern Standard Arabic (MSA), have encountered limitations in effectively addressing this task. However, advancements in NLP, particularly the introduction of specialized models such as AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020), have provided new opportunities for improving hate speech detection in Arabic.

AraBERT, initially employed for hate speech detection, was predominantly trained on MSA data. While MSA serves as a formal written form of Arabic, it does not adequately capture the linguistic nuances and variations present in different dialects and informal online communication. Consequently, the performance of AraBERT in detecting hate speech, particularly in social media contexts where dialectal Arabic dominates, has been suboptimal.

In contrast, MarBERT, a model specifically trained on dialectal Arabic data, has shown more promising results in hate speech detection. By leveraging a broader range of dialectal data, MarBERT can better capture the intricacies and nuances of Arabic dialects commonly used in social media platforms. This enhanced understanding of dialectal Arabic enables MarBERT to effectively identify hate speech in texts written in these dialects.

Social media platforms have emerged as significant sources of data for hate speech detection in Arabic. Given the informal nature of communication on these platforms, which often reflects the diverse dialectal usage, models like MarBBERT that have been trained on dialectal data prove to be more successful in accurately classifying hate speech instances. The utilization of MarBERT in hate speech detection demonstrates the importance of tailoring models to the specific linguistic characteristics and contextual environments in which they will be applied.

To evaluate and compare the performance of hate speech detection models in Arabic, the OSACT5 (Mubarak et al., 2022) benchmark has gained recognition. OSACT5 provides a standardized evaluation framework for assessing the effectiveness of different models in identifying hate speech across multiple dialects and contexts. By adopting the OSACT5 benchmark, researchers and practitioners in the field

can benchmark their hate speech detection systems, fostering advancements in the field and enabling fair comparisons of different approaches.

#### 2.1.1.3 Progression of NLP

Over the years, NLP has witnessed remarkable progress, revolutionizing machines' comprehension, interpretation, and production of the human language. Commencing with rule-based systems to statistical methods and progressing towards cutting-edge deep learning methods, NLP has profoundly shaped our engagement with technology. Currently, it is a vital component in various applications, including search engines, virtual assistants, and machine translation, enhancing our overall digital experience.

**Rule-based Systems** In the early days of NLP, rule-based systems were the primary approach for understanding and generating human language. These systems rely on handcrafted rules, often created by linguists, to process and analyze texts (Pereira and Warren, 1980). One of the most well-known rule-based systems is the ELIZA chatbot developed in the 1960s by Joseph Weizenbaum (Weizenbaum, 1966). ELIZA uses pattern matching and substitution rules to hold conversations with users, simulating a Rogerian psychotherapist. The success of ELIZA inspired researchers to develop more sophisticated rule-based systems, such as SHRDLU, a system designed by Terry Winograd that could interpret and respond to natural language commands in a limited blocks-world domain (Winograd, 1972).

Another milestone in rule-based NLP was the development of the first expert systems, such as LUNAR, which provided natural language interfaces to the databases (Woods, 1973). These expert systems rely on domain-specific knowledge encoded in the form of production rules and semantic networks (Quillian, 1968). Despite the initial success of rule-based systems, they have several limitations, including their inability to scale to the complexity and variability of human language, and the labor-intensive process of crafting rules (Chomsky, 1957). Consequently, researchers have begun exploring statistical methods to address these issues and develop more robust NLP systems.

**Statistical Methods** Statistical approaches in NLP emerged in the late 1980s and early 1990s, leveraging statistical models to learn linguistic patterns from data

(Manning and Schütze, 1999). This paradigm shift marked a departure from the handcrafted rules of the previous era as researchers started focusing on data-driven techniques to capture the intricacies of human language (Charniak et al., 1993).

One of the key advancements in this era was the development of the Hidden Markov Model (HMM) for part-of-speech tagging, a fundamental NLP task (Rabiner, 1989). HMMs are generative probabilistic models that describe the underlying sequence of part-of-speech tags in a sentence. These models were used to predict the most likely sequence of tags for a given sentence, providing a more robust and data-driven approach than rule-based systems (Church, 1988). HMMs were later extended to other sequence-labeling tasks, such as named entity recognition (Bikel et al., 1997) and shallow parsing (Ramshaw and Marcus, 1995).

Another major breakthrough was the introduction of probabilistic context-free grammar (PCFGs) for parsing, enabling the modeling of syntactic structures in a more principled manner (Booth and Thompson, 1973). PCFGs are probabilistic extensions of context-free grammars that assign probabilities to individual production rules (Jelinek, 1985). These probabilities are learned from annotated data, allowing PCFGs to capture the distribution of syntactic structures in a given language (Charniak, 1997). PCFG-based parsers, such as the Collins parser (Collins, 1997) and Charniak parser ((Charniak, 2000), were among the most influential parsing models of the statistical NLP era.

During this period, researchers developed various machine learning algorithms, such as the Maximum Entropy classifier (Berger et al., 1996), Support Vector Machines (Cortes and Vapnik, 1995), and Decision Trees (Quinlan, 1986), for tasks such as text classification, sentiment analysis (Pang et al., 2002), and relation extraction (Kambhatla, 2004). These algorithms demonstrate the potential of statistical methods for a wide range of NLP tasks and lay the foundation for the subsequent rise of deep learning techniques.

**Deep Learning Techniques** The advent of deep learning has revolutionized NLP, leading to remarkable improvements in performance across various tasks (Goodfellow et al., 2016). Deep learning models, such as neural networks, learn hierarchical representations of data, allowing them to capture complex patterns

and dependencies in the human language (Bengio et al., 2013). This section discusses some of the key deep-learning models and architectures that have had a significant impact on NLP research and applications.

The introduction of word embeddings such as Word2Vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) marked a significant shift in the way textual data were represented. Word embeddings are continuous vector representations that capture the semantic and syntactic relationships between words and have become an essential building block for many deep learning-based NLP models. These embeddings enable neural networks to learn more expressive representations of text, improving performance on tasks such as sentiment analysis, machine translation, and text classification.

Recurrent Neural Networks (RNNs) and their variants, such as Long Short-Term Memory (LSTM) networks (Hochreiter and Schmidhuber, 1997) and Gated Recurrent Units (GRUs) (Cho et al., 2014), have been instrumental in modeling sequences and temporal dependencies in NLP. RNNs have been applied to a wide range of tasks, including language modeling (Mikolov et al., 2010), machine translation (Sutskever et al., 2014), and text summarization (Rush et al., 2015).

The introduction of attention mechanisms, first proposed by (Bahdanau et al., 2014), further transformed NLP models, leading to significant performance improvements in tasks, such as machine translation and text summarization. Attention mechanisms allow models to dynamically weigh the importance of different parts of the input sequence, enabling them to capture long-range dependencies and contextual information better (Vaswani et al., 2017).

The Transformer architecture introduced by (Vaswani et al., 2017) has become the foundation for many state-of-the-art NLP models. Transformers leverage self-attention mechanisms to model complex dependencies in text, allowing for more efficient parallelization during training and inference compared with RNNs. Transformers have been applied to a wide range of tasks including machine translation (Vaswani et al., 2017), text classification (Devlin et al., 2019), and question answering (Yang et al., 2019).

Pretraining and transfer learning have emerged as powerful techniques for training NLP models, enabling them to leverage large amounts of unlabeled data and adapt to new tasks with limited labeled data (Ruder et al., 2019). Models such as BERT (Devlin et al., 2019), GPT-2 (Radford et al., 2019), and RoBERTa (Liu et al.,

2019) have demonstrated the potential of pretraining on tasks such as language modeling, and have achieved state-of-the-art performance across a wide range of NLP benchmarks, such as the GLUE benchmark (Wang et al., 2018) and the SuperGLUE benchmark (Wang et al., 2019).

#### 2.1.1.4 Transformers

The Transformer is a deep learning architecture introduced by (Vaswani et al., 2017). It has revolutionized various NLP tasks, such as machine translation, text summarization, and language understanding. The key innovation of Transformers lies in their attention mechanism, which allows them to capture long-range dependencies efficiently.

The basic structure of a Transformer consists of an encoder and a decoder. Both the encoder and decoder are composed of multiple layers, and each layer is further divided into sub-layers. The sub-layers typically include a multi-head self-attention mechanism and a position-wise feed-forward neural network.

The self-attention mechanism is the core component of Transformers. It allows the model to weigh different input elements (e.g., words in a sentence) when making predictions. In simple terms, self-attention measures the importance of each element in a sequence by comparing it with all other elements.

To perform self-attention, each input element is associated with three vectors: query, key, and value. These vectors are linear projections of the original input and enable the model to determine the importance of an element by comparing it with others.

To calculate attention scores, the model computes the dot product between a query vector and the transposed key vectors of all other elements in the sequence. These dot products are scaled by the square root of the dimension of the key vectors. The resulting scores are then passed through a softmax function to obtain attention weights. This is illustrated in figure 2.1 (Alammar, 2018)

The attention weights obtained in the previous step determine how much each value vector contributes to the representation of the current element. The weighted sum of the value vectors, using the attention weights as coefficients, yields the final attended representation of the element.

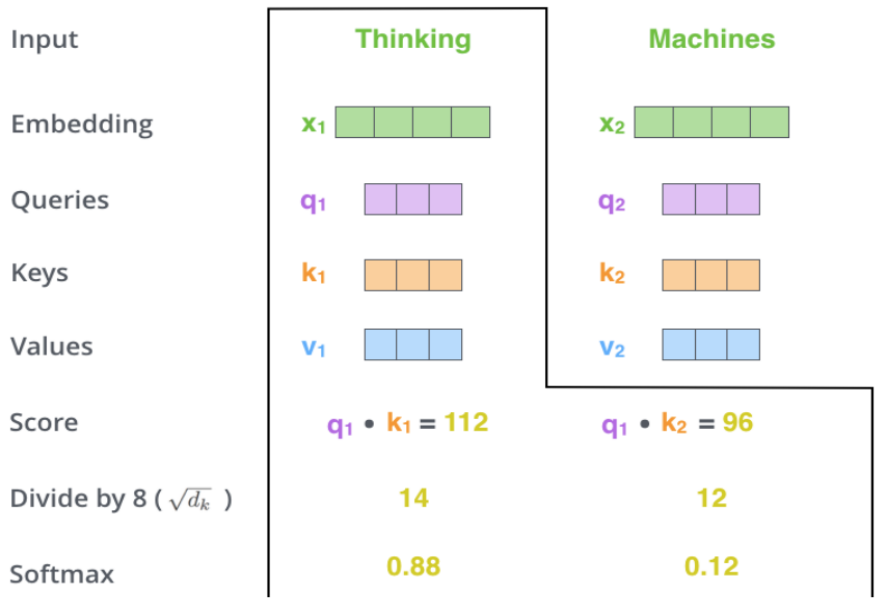



FIGURE 2.1: Illustration of self-attention calculation in transformer

In Transformers, self-attention is typically performed multiple times in parallel, known as multi-head attention. Each attention head has its own set of queries, keys, and values and focuses on different relationships in the input, allowing the model to capture diverse information. The multi-head attention mechanism results in a weighted combination of the attention heads.

Since Transformers lack sequential information inherently present in recurrent models, they require positional encoding. Positional encoding assigns each element in the sequence a unique positional embedding, providing information about its position or order. One commonly used formulation for positional encoding is based on sine and cosine functions. The positional encoding is added to the original input embeddings, allowing the model to incorporate positional information into the self-attention mechanism.

After the self-attention mechanism, the output is passed through a position-wise feed-forward neural network. This network consists of two linear layers separated by a non-linear activation function, such as ReLU. It aims to capture complex interactions between elements in the sequence.

To stabilize training and facilitate information flow, Transformers employ residual connections and layer normalization. Residual connections allow the model to learn incremental changes, and layer normalization helps normalize the outputs of each sub-layer.

In tasks like machine translation, Transformers utilize an encoder-decoder architecture. The encoder processes the input sequence, while the decoder generates the output sequence. Both encoder and decoder share similar architectures but have some differences in the way they are trained and used.

During training the decoder, an additional component called masked self-attention is used. It prevents the decoder from attending to future positions in the sequence, ensuring the model generates the output sequentially and avoids information leakage.

In the decoder, an additional positional encoding is used to incorporate information about the relative positions of the input and output elements. This helps the decoder generate output sequences by attending to relevant parts of the input sequence.

This complexity added by transformers has elevated the performance of various NLP tasks, such as machine translation, sentiment analysis, capturing social meaning, and text generation, pushing the boundaries of what was previously achievable in language understanding and generation.

#### 2.1.1.5 Tokenization Methods

Tokenization is an essential step in NLP that involves splitting text into individual tokens or subword units. There are several popular methods for tokenization, including BPE (Byte Pair Encoding) (Sennrich et al., 2015), byte-BPE (Wang et al., 2020), WordPiece (Kudo, 2018b), unigram (Kudo, 2018a), and SentencePiece (Kudo and Richardson, 2018).

**BPE** is a subword tokenization technique that segments words into subword units based on their frequency in a given corpus. It starts with treating each word as a sequence of characters and initializes a vocabulary with individual characters. It then iteratively merges the most frequent character pair to form a new subword unit until reaching a predefined vocabulary size or convergence criterion. BPE is effective in handling out-of-vocabulary (OOV) words and rare words since it can break down words into subword units. For example, the word "unhappiness" can be tokenized as "un" + "happi" + "ness". BPE has been widely used in various NLP tasks, including machine translation and text generation.

**Byte-BPE** is a variant of BPE that operates at the byte level rather than the character level. It allows the encoding of multi-byte characters as a single token. Byte BPE is particularly useful when dealing with languages that use multi-byte character encodings, such as UTF-8. By operating at the byte level, Byte BPE avoids splitting multi-byte characters into separate tokens. This approach ensures that each character is treated as a single unit, even if it consists of multiple bytes. Byte BPE is commonly used in multilingual models and has shown benefits for handling diverse languages.

**WordPiece** tokenization is another subword tokenization technique that segments words into subword units. It is similar to BPE but differs in the merging strategy. Instead of merging character pairs, WordPiece uses a predefined vocabulary of whole words as well as subword units. WordPiece tokenization starts with a vocabulary of individual characters and words. It then greedily merges the most frequent pair of units until reaching the desired vocabulary size. The resulting vocabulary contains a combination of whole words and subword units. WordPiece is widely used in models like BERT and has become a standard tokenization method for many NLP tasks.

**Unigram** tokenization is a statistical tokenization method that uses a language model to predict the likelihood of different token boundaries within a text. It aims to find the most probable segmentation of the text into tokens based on their frequency. Unigram tokenization considers the entire text as a sequence of characters or bytes and applies dynamic programming to find the optimal segmentation. By maximizing the overall likelihood of the tokens, it identifies the most probable boundaries between them. Unigram tokenization can be effective in capturing word boundaries in languages without explicit delimiters.

**SentencePiece** is a more flexible tokenization method that can handle both word-level and subword-level tokenization. It allows for fine-grained control over the vocabulary size and tokenization granularity. SentencePiece employs an unsupervised training algorithm that can be based on BPE, unigram, or other methods. It operates on the subword level and can handle various languages with different writing systems. SentencePiece has gained popularity due to its versatility and ease of integration into NLP pipelines.

These different methods for tokenization provide various trade-offs in terms of granularity, handling of OOV words, and computational efficiency. The choice of tokenization method depends on the specific NLP task, dataset characteristics, and the requirements of the downstream application. It's common for different methods to be used in different contexts, and experimentation is often necessary to determine the most suitable tokenization approach for a particular use case.

#### 2.1.1.6 BERT

BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019) is a groundbreaking model that has revolutionized the field of NLP. By utilizing a bidirectional context, it brought about a substantial change by enabling it to capture rich semantic information from both the words that came before and after it in a sentence. BERT's architecture, based on the Transformer model, has proven to be highly effective in various NLP tasks.

BERT operates in two main stages: pre-training and fine-tuning. During pre-training, the model learns from a massive amount of unlabeled text by predicting the masked words in a sentence. This masked language-modeling objective enables BERT to develop a deep understanding of contextual word representations. Additionally, BERT learns a task called "next sentence prediction," which helps it comprehend the relationship between two consecutive sentences.

BERT's architecture consists of a stack of Transformer layers. Each layer incorporates self-attention mechanisms, allowing the model to attend to different parts of the input sequence during processing. This attention mechanism facilitates the capture of global dependencies and enables BERT to effectively model long-range dependencies in the text. The bidirectional nature of BERT training enables it to excel in tasks requiring contextual understanding, such as machine translation, named entity recognition, sentiment analysis, and question answering.

BERT's true power lies in its ability to be fine-tuned on specific downstream tasks. By adding a task-specific layer on top of the pre-trained BERT model, the entire architecture can be fine-tuned using labeled data from the target task. BERT has achieved remarkable performance in various classification tasks due to its capability as an encoder. Its ability to generate high-quality contextualized word embeddings provides rich representations for the downstream classifiers.

In classification tasks, BERT has demonstrated exceptional performance. It has excelled in sentiment analysis by accurately capturing sentiment nuances and contextual cues from the text. In addition, BERT has proven to be highly effective in text classification tasks, such as spam detection, intent recognition, and hate speech detection. By leveraging bidirectional context and deep contextual understanding, BERT can capture intricate linguistic patterns and semantic relationships, leading to improved classification accuracy.

Overall, BERT's introduction has brought a paradigm shift in NLP by introducing the bidirectional context and leveraging the power of Transformers. Its ability to pre-train on massive amounts of unlabeled text and subsequently fine-tune on specific tasks has made it a go-to model for a wide range of NLP applications. BERT's architecture and encoding capabilities have enabled breakthroughs in various classification tasks, showcasing the impact and potential of advanced language representation models in the field of NLP.

#### 2.1.1.7 AraBERT and MarBERT

AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020) are significant models in the field of NLP that leverage the power of the BERT architecture specifically for the Arabic language. Both models have been pre-trained on large-scale Arabic datasets, enabling them to capture the linguistic complexities specific to Arabic text.

**Pre-training**

Pre-training is a crucial step for both AraBERT and MarBERT. During pretraining, the models learn from massive amounts of unlabeled Arabic text. The training data includes diverse sources such as web content, news articles, books, and Wikipedia, ensuring a wide coverage of Arabic language usage. This pre-training phase helps the models develop rich representations of Arabic words and their contextual meanings.

AraBERT is pre-trained mainly on MSA. The datasets AraBERT was pre-trained on are:

- **Arabic Corpus El Khair:** (El-Khair, 2016) This contemporary corpus comprises approximately 1.5 billion words and encompasses a vast collection

of more than 5 million articles. These articles have been sourced from ten prominent news outlets, covering a diverse range of topics and representing a broad spectrum of journalistic writing styles. The corpus spans eight countries, ensuring a comprehensive representation of Arabic language usage across various contexts.

- **OSIAN (Open Source International Arabic News Corpus):** (Zeroual et al., 2019) This is an extensive collection of Arabic news articles, totaling around 3.5 million in number. The corpus encompasses articles from a wide range of international Arabic news sources. It offers a diverse and comprehensive dataset, enabling AraBERT to capture the nuances of Arabic language usage across different regions and news domains.

On the other hand, MarBERT was trained on a combination of MSA and dialectal Arabic data. Dialectal Arabic refers to the regional variations and colloquial forms of the language. MarBERT's training data included a large-scale in-house dataset consisting of approximately 6 billion Arabic tweets. From this extensive dataset, a random sampling approach was employed to extract a subset of 1 billion Arabic tweets. This random sampling process ensured the inclusion of diverse linguistic patterns, topics, and user-generated content representative of the Arabic language in social media contexts. MarBERT is also pre-trained without the next sentence prediction (NSP) objective since tweets are short.

The inclusion of dialectal Arabic data in MarBERT's training corpus gives it an advantage in tasks where the target text is predominantly written in dialects, such as social media analysis. Since most user-generated content, including social media posts, is written in dialectal Arabic, MarBERT's exposure to this type of data helps it better understand and process the nuances and variations present in these texts.

While Arabert performs well in tasks related to Modern Standard Arabic, MarBERT often outperforms Arabert in dialect-related tasks and social media analysis. MarBERT's training on dialectal data allows it to capture the specific characteristics of informal Arabic, including dialectal spelling, slang, and unique linguistic patterns. This enables MarBERT to achieve better results in dialect identification, sentiment analysis on social media, and other tasks where dialectal Arabic plays a crucial role.

**Tokenization**

AraBERT utilizes the SentencePiece tokenizer (Kudo and Richardson, 2018) in unigram (Kudo, 2018a) mode in conjunction with the Farasa preprocessing toolkit (Abdelali et al., 2016). The SentencePiece tokenizer is a widely adopted method that operates on subword units. It serves the primary objective of decomposing Arabic words into their constituent components, including prefixes, stems, and suffixes,allowing for a more flexible and efficient representation of the language. This process, known as Arabic word segmentation, holds significant importance as a fundamental pre-processing step for numerous Arabic NLP tasks.

This tokenizer is particularly beneficial for Arabic text, which exhibits complex morphology and rich linguistic structures. The Farasa preprocessing toolkit, on the other hand, is specifically designed for Arabic text processing and provides various linguistic analysis functionalities such as tokenization, segmentation, and morphological analysis. By incorporating the Farasa preprocessing toolkit, AraBERT ensures accurate and linguistically-aware representation of Arabic text during the tokenization process.

In contrast, MarBERT employs the WordPiece (Kudo, 2018b) tokenizer without utilizing the Farasa preprocessing toolkit. The WordPiece tokenizer is a widely used subword tokenization method that breaks down words into smaller subword units based on their statistical properties. While not specifically tailored for Arabic, the WordPiece tokenizer has proven effective in capturing meaningful subword units in various languages. By adopting the WordPiece tokenizer, MarBERT can handle Arabic text without the explicit use of the Farasa preprocessing toolkit. This approach allows MarBERT to bypass the additional processing steps associated with Farasa, potentially simplifying the tokenization pipeline.

The choice of tokenizer and preprocessing approach impacts the representation of Arabic text in both AraBERT and MarBERT. AraBERT's utilization of the SentencePiece tokenizer along with the Farasa preprocessing toolkit enhances its ability to handle Arabic linguistic complexities, such as morphology and syntactic structures. This approach enables AraBERT to capture fine-grained linguistic details and produce more accurate representations for downstream NLP tasks. On the other hand, MarBERT's use of the WordPiece tokenizer offers a more streamlined tokenization process, potentially leading to increased efficiency and reduced computational overhead.

It is important to note that the choice of tokenizer and preprocessing strategy may depend on the specific requirements of the NLP task at hand. While AraBERT benefits from the comprehensive linguistic analysis provided by the Farasa preprocessing toolkit, MarBERT's simplified tokenization approach may be advantageous in scenarios where computational efficiency is a priority.

### 2.1.2 Adversarial Attacks on Deep Learning

Deep learning has significantly advanced the state of the art in various domains, including natural language processing (NLP) and computer vision. However, these models have been found vulnerable to adversarial attacks, which pose a significant threat to their security and robustness. This section provides a comprehensive study of adversarial attacks on deep learning models, focusing on NLP, and reviews the concepts, foundations, components, and properties of adversarial attacks in NLP.

Adversarial attacks are designed to manipulate deep learning models by introducing small, imperceptible perturbations to the input data, leading to incorrect predictions or outputs (Szegedy et al., 2013). These perturbed examples, known as adversarial examples, are crafted explicitly to deceive the model while maintaining their perceptual similarity to the original, unaltered inputs.

#### 2.1.2.1 Attacker's Goal

**2.1.2.1.1 Misclassification (Untargeted Attack)** The attacker's primary goal is often to cause a model to produce incorrect outputs. For instance, an attacker may attempt to manipulate a sentiment analysis model to classify positive reviews as negative or vice versa.

**2.1.2.1.2 Targeted Misclassification (Targeted Attack)** In some cases, the attacker may desire a specific, erroneous output. For instance, they might want a spam detection model to classify a particular email as legitimate, even if it is spam.

**2.1.2.1.3 Confidence Reduction** The attacker aims to reduce the model's confidence in its predictions, even if the output remains correct. This may lead to a decrease in the model's overall performance and trustworthiness.

#### 2.1.2.2 Attacker's Knowledge

**2.1.2.2.1 White-Box Attacks (Full-Knowledge Attacks)** The attacker has complete knowledge of the model, including its architecture, parameters, and training data (Papernot et al., 2016b). This allows the attacker to generate highly effective adversarial examples by leveraging the model's inherent weaknesses.

**2.1.2.2.2 Grey-Box Attacks (Limited-Knowledge Attacks)** The attacker has limited knowledge of the model, such as its architecture or some of its parameters, but does not have access to the training data or the complete model (Bhagoji et al., 2018). While the attacker may not have access to the exact model weights, they can still create adversarial examples based on their understanding of the model's behavior.

**2.1.2.2.3 Black-Box Attacks (Zero-Knowledge Attacks)** The attacker has no knowledge of the model's internals and can only access its inputs and outputs (Chen et al., 2017). Despite this limited knowledge, attackers can still generate effective adversarial examples by exploiting transferability, where adversarial examples generated for one model are also effective on similar models.

#### 2.1.2.3 Taxonomy of Adversarial Attacks in NLP

Adversarial attacks in NLP can be broadly organized into the following five categories: textual perturbation attacks, semantic manipulation attacks, transferability attacks, adversarial training data generation, and combination attacks.

**2.1.2.3.1 Textual Perturbation Attacks** Textual perturbation attacks involve adding, deleting, or modifying words, phrases, or characters in the input text to deceive the target model. These attacks can be further divided into three subcategories: word-level attacks, character-level attacks, and paraphrase-based attacks.

**2.1.2.3.1.1 Word-Level Attacks** These attacks replace words in the input text with semantically similar substitutes to mislead the target model while preserving the overall meaning. Example methods include TextFooler (Jin et al., 2020) and the HotFlip attack (Ebrahimi et al., 2018). In addition, (Alzantot et al., 2018b) proposed the Genetic Algorithm-based attack, which iteratively applies word-level perturbations to generate adversarial examples.

**2.1.2.3.1.2 Character-Level Attacks** These attacks perturb the input text at the character level, for example, by applying misspellings or typos. DeepWordBug (Gao et al., 2018b) is an example of such an attack. Another approach called the "adversarial keyboard" was proposed by (Grosse et al., 2017), which maps clean inputs to adversarial ones using a predefined keyboard layout.

**2.1.2.3.1.3 Sentence-Level Attacks** In these attacks, the input sentence is rephrased while preserving its original meaning to deceive the target model. The PAWS dataset (Zhang et al., 2019) and the ParaBank dataset (Hu et al., 2019) provide examples of paraphrase-based attacks. Additionally, (Zhao et al., 2018) proposed a paraphrase-based attack using reinforcement learning for generating adversarial examples.

**2.1.2.3.2 Semantic Manipulation Attacks** Semantic manipulation attacks exploit the limitations of NLP models in understanding the semantics of the input text. Some examples of semantic manipulation attacks include:

**2.1.2.3.2.1 Universal Adversarial Triggers** In these attacks, a fixed sequence of words is added to the input text to trigger a specific, incorrect output. The Universal Adversarial Triggers method proposed by(Wallace et al., 2019) is a seminal example of this type of attack.

**2.1.2.3.2.2 Adversarial Question-Answering Attacks** These attacks generate questions that require a deep understanding of the text to answer correctly, thereby exploiting the limitations of question-answering models. The Adversarial SQuAD dataset (Jia and Liang, 2017) provides examples of such attacks. Moreover, (Jia et al., 2020) developed AdversarialQA, a dataset with adversarial examples specifically designed for question-answering systems.

**2.1.2.3.2.3 Trojan Attacks** In these attacks, the target model is trained to associate specific trigger phrases with incorrect outputs. The TextTrojan method (Chen et al., 2020) is an example of this type of attack.

**2.1.2.3.3 Transferability Attacks** Transferability attacks exploit the property that adversarial examples generated for one model can also be effective against similar models. This property allows attackers to launch black-box attacks on NLP models by generating adversarial examples using surrogate models. The black-box attack on sentiment analysis models proposed by (Cheng et al., 2020) is an example of a transferability attack. (Papernot et al., 2017) also demonstrated the transferability of adversarial examples across different NLP tasks.

**2.1.2.3.4 Adversarial Training Data** Adversarial training data generation includes methods that create training data with adversarial properties to improve model robustness. These attacks can be viewed as a form of defense against adversarial attacks.

**2.1.2.3.4.1 Adversarial Training** (Goodfellow et al., 2014b) proposed adversarial training, which augments the training dataset with adversarial examples in order to make the model more robust to adversarial attacks. This method has been applied to various NLP tasks, such as text classification and machine translation (Miyato et al., 2017).

**2.1.2.3.4.2 Data Poisoning** Data poisoning attacks inject adversarial examples into the training dataset to manipulate the target model's behavior. (Steinhardt et al., 2017) provided a comprehensive analysis of data poisoning attacks and their implications for model training.

**2.1.2.3.5 Combination Attacks** Combination attacks involve multiple attack strategies or exploit multiple vulnerabilities in the target model. For example, (Li et al., 2021) proposed a combination attack that generates adversarial examples by simultaneously perturbing the input text at the word and character levels.

**2.1.2.3.6 Information Retrieval Attacks** These attacks involve exploiting the inherent biases or vulnerabilities of the target model without necessarily generating adversarial examples. However, their objective is to retrieve or extract information from or about the model.

**2.1.2.3.6.1 Model Inversion Attacks** In this type of attack, the attacker aims to recover sensitive information about the training data or individual instances by analyzing the model's outputs. This can lead to privacy breaches and the disclosure of sensitive information.

**2.1.2.3.6.2 Model Extraction Attacks** Model extraction attacks involve extracting sensitive information or replicating the functionality of a machine learning model by strategically querying the model and building a surrogate model. Adversaries aim to infer the model's architecture, hyperparameters, and training data through iterative queries.

**2.1.2.3.6.3 Membership Inference Attacks** These attacks attempt to determine whether a given data instance was part of the target model's training set by analyzing its output. Successful membership inference attacks can also lead to privacy breaches and the disclosure of sensitive information.

#### 2.1.2.4 Attack Techniques

**2.1.2.4.1 Gradient-Based Attacks** Gradient-based attacks leverage the gradients of the target model's loss function with respect to the input to generate adversarial examples (Szegedy et al., 2013). These attacks require access to the model's architecture and parameters, making them white-box attacks.

**2.1.2.4.2 Optimization-Based Attacks** Optimization-based attacks search for adversarial examples by solving optimization problems, often using iterative methods or zeroth-order optimization (Chen et al., 2017). These attacks can be either white-box or black-box, depending on the information available about the target model.

**2.1.2.4.3 Rule-Based Attacks** Rule-based attacks use linguistic rules or heuristics to generate adversarial examples. These attacks often involve replacing words or phrases with synonyms, reordering words, or inserting irrelevant phrases (Ebrahimi et al., 2017). Rule-based attacks are generally black-box attacks and can be more interpretable than gradient-based or optimization-based attacks.

**2.1.2.4.4 Generative Adversarial Networks** Generative adversarial networks (GANs) consist of a generator and a discriminator model. The generator creates adversarial examples, while the discriminator attempts to distinguish between real and adversarial examples (Goodfellow et al., 2014a). GANs can be used to generate adversarial examples that are harder to detect by defense mechanisms.

**2.1.2.4.5 Reinforcement Learning-Based Attacks** Reinforcement learning-based attacks use reinforcement learning algorithms to generate adversarial examples by learning a policy that maximizes the reward, which is usually defined as the probability of causing misclassification or incorrect predictions (Alzantot et al., 2018b).

**2.1.2.4.6 Black-Box Attacks** Black-box attacks can use a variety of techniques, including transferability-based attacks, where adversarial examples generated for one model are used to attack another model, and query-based attacks, where the attacker iteratively queries the target model to obtain information for generating adversarial examples (Chen et al., 2017).

#### 2.1.2.5 Defensive Strategies Against Adversarial Attacks

Several defense techniques have been proposed to mitigate the impact of adversarial attacks on deep learning models. In this section, we discuss some of the most prominent methods.

**2.1.2.5.1 Adversarial Training** Adversarial training is a popular defense technique that involves augmenting the training data with adversarial examples (Goodfellow et al., 2014b). This approach helps the model learn robust features that are less sensitive to adversarial perturbations.

**2.1.2.5.2 Defensive Distillation** Defensive distillation is a defense technique that trains a model to produce soft labels, i.e., probabilistic outputs, instead of hard labels (Papernot et al., 2016a). This process makes it more challenging for the attacker to compute gradients and craft adversarial examples.

**2.1.2.5.3 Gradient Masking** Gradient masking is a defense technique that aims to obfuscate the gradient information of the model, making it difficult for the attacker to compute adversarial perturbations (Papernot et al., 2016a). This is typically achieved by adding non-differentiable components to the model or by approximating the gradients.

**2.1.2.5.4 Randomized Input Transformations** Randomized input transformations apply random transformations to the input data before feeding it to the model, making it difficult for the attacker to craft effective adversarial examples (Xie et al., 2017). Examples of such transformations include image cropping, resizing, and rotation.

**2.1.2.5.5 Certified Defenses** Certified defenses provide mathematical guarantees on the model's robustness against adversarial perturbations (Cohen et al., 2019). These defenses typically involve computing an upper bound on the adversarial perturbation or applying formal verification techniques to ensure the model's robustness.

**2.1.2.5.6 Ensemble-based Approaches** Ensemble-based approaches combine the predictions of multiple models to improve the overall robustness against adversarial attacks. These approaches can include model averaging, model stacking, or using diverse model architectures (Papernot et al., 2016a).

#### 2.1.2.6 Evaluation Aspects of Adversarial Attacks

Evaluating adversarial attacks is crucial for understanding model vulnerabilities, developing robust defenses, establishing benchmarks, and guiding model selection and deployment in security-sensitive applications. By assessing the effectiveness and impact of adversarial attacks, researchers and practitioners can identify weak

points in machine learning systems, devise mitigation strategies, and foster innovation in the field, ultimately contributing to the development of secure and reliable machine learning solutions.

In this section, we will explore various evaluation aspects of adversarial attacks, including perceptibility, validity, naturality, robustness, transferability, efficiency, and scalability.

**2.1.2.6.1 Perceptibility** refers to the degree to which adversarial perturbations are noticeable by humans or other algorithms. The imperceptibility of adversarial examples is a crucial aspect of their success, as it allows them to deceive both human observers and machine learning models alike (Goodfellow et al., 2015).

In many cases, adversarial perturbations are so small that they are indistinguishable from the original input. For example, an image of a cat can be subtly altered to be misclassified as a dog by a deep learning model, while remaining visually indistinguishable to a human observer (Athalye et al., 2018). This imperceptibility can be quantified using measures such as the $L_p$ norm, which calculates the magnitude of the perturbations in relation to the original input (Xu et al., 2019).

**2.1.2.6.2 Validity** refers to the effectiveness of adversarial examples at causing misclassifications in their target model. An adversarial example is considered valid if it successfully deceives the model into producing an incorrect output. To assess the validity of adversarial attacks, researchers typically use metrics such as attack success rate, which calculates the proportion of adversarial examples that successfully cause misclassifications (Carlini and Wagner, 2017).

The validity of adversarial attacks depends on various factors, including the choice of attack algorithm, the target model's architecture, and the strength of the perturbations. For instance, the Fast Gradient Sign Method (FGSM) (Goodfellow et al., 2015) is a widely-used attack algorithm that generates adversarial examples by calculating the gradient of the model's loss function with respect to the input. However, the effectiveness of FGSM can be limited by factors such as the choice of step size and the robustness of the target model (Moosavi-Dezfooli et al., 2016).

**2.1.2.6.3 Naturality** refers to the degree to which adversarial examples resemble real-world data samples. Ideally, adversarial examples should not only be

imperceptible but also appear natural to both human observers and machine learning models. This ensures that the adversarial examples are not easily detected by defense mechanisms designed to identify and reject unnatural inputs (Hendrycks et al., 2021).

To craft natural adversarial examples, researchers have proposed various techniques such as using generative adversarial networks (GANs) (Samangouei et al., 2018), or optimizing the adversarial perturbations to minimize perceptual distance (Sharma and Chen, 2018). These methods aim to generate adversarial examples that lie within the manifold of natural data samples, making them more difficult to detect and mitigate.

**2.1.2.6.4 Robustness** refers to the ability of a machine learning model to withstand adversarial attacks. A robust model can still produce correct outputs even in the presence of adversarial perturbations. To improve the robustness of models against adversarial attacks, researchers have proposed various defense techniques such as adversarial training (Madry et al., 2018), which involves training the model on a mix of clean and adversarial examples.

However, achieving robustness against adversarial attacks remains an open challenge due to the inherent limitations of current machine learning models and the ongoing arms race between attack and defense techniques. For instance, many defense mechanisms have been shown to be vulnerable to adaptive attacks, where the attacker is aware of the defense strategy and crafts adversarial examples accordingly (Athalye et al., 2018).

**2.1.2.6.5 Transferability** refers to the ability of adversarial examples to cause misclassifications across different models or model architectures. This property is particularly concerning because it suggests that an attacker can craft adversarial examples targeting one model and potentially use them to attack other models without any knowledge of their architecture or parameters (Papernot et al., 2016a).

Transferability is believed to arise due to the presence of shared decision boundaries among different models and the use of high-dimensional input spaces, which makes it difficult for models to learn completely distinct representations of the data (Tramèr et al., 2017). To mitigate the transferability of adversarial attacks, researchers have proposed techniques such as input transformations (Xu et al.,

2017) and ensemble adversarial training (Tramèr et al., 2017), which aim to increase the diversity of model defenses and reduce their vulnerability to transferable attacks.

**2.1.2.6.6 Efficiency** refers to the computational cost and speed of generating adversarial examples. In practical settings, an attacker may have limited time and resources, making the efficiency of adversarial attacks a crucial factor in their success. Several attack algorithms have been proposed that prioritize efficiency, such as the aforementioned FGSM (Goodfellow et al., 2015), which generates adversarial examples using a single step.

However, the trade-off between efficiency and attack success rate must be carefully considered, as more efficient attacks may not always be the most effective. For instance, iterative attack algorithms such as the Projected Gradient Descent (PGD) method (Madry et al., 2018) can achieve higher success rates than single-step methods like FGSM, but at the cost of increased computational complexity.

**2.1.2.6.7 Scalability** Scalability refers to the ability of adversarial attacks to be applied to large-scale machine-learning systems with diverse architectures and input modalities. Adversarial attacks have been demonstrated to be effective against various types of models, including image classifiers (Szegedy et al., 2013), speech recognition systems (Carlini and Wagner, 2018), and natural language processing models (Jin et al., 2020). This broad applicability underscores the significance of adversarial attacks as a pervasive threat to machine learning systems.

To address the scalability of adversarial attacks, researchers have proposed various attack and defense techniques that can be adapted to different model architectures and input modalities (Akhtar and Mian, 2018). However, the ongoing arms race between attackers and defenders, coupled with the rapid evolution of machine learning models and applications, necessitates continuous research and innovation to ensure the security and robustness of these systems.

When evaluating adversarial attacks, it is essential to consider these metrics in combination, as they provide complementary insights into the attack's effectiveness, subtlety, versatility, and practicality. By understanding the performance

of adversarial attacks on these dimensions, researchers can develop more effective defense strategies and ensure the robustness and security of machine learning systems.

## 2.2 Related Work

The field of adversarial attacks in NLP is experiencing significant growth, driven by the widespread use of deep learning models and their increasing accessibility to the public. While researchers have extensively studied the impact of adversarial attacks on NLP models, the focus has primarily been on commonly used languages such as English, French, and others. However, the attention given to the Arabic language in this context has been relatively limited, with only a few works addressing it.

This section will go over the attacks implemented in other languages that we were inspired by and have continued to adapt to the Arabic domain. Furthermore, we present the Arabic-language attacks that we worked on to overcome the absence of organizational structure seen in previous attacks in other languages. By expanding on past research, we hope to draw attention to the particular problems and complexities of adversarial assaults in Arabic, contributing to a more thorough knowledge of NLP security across multiple linguistic contexts.

### 2.2.1 Attacks on Other Languages

#### 2.2.1.1 From Hero to Zéroe (Eger and Benz, 2020)

In this paper the authors present a comprehensive benchmark to evaluate the effectiveness of low-level adversarial attacks on deep neural networks. The main contribution of this work lies in the creation of Zéroe, a dataset that systematically assesses adversarial examples generated by various attack methods, such as character flipping and character insertion, across different model architectures, tasks, and datasets. This benchmark provides a solid foundation for understanding the robustness of deep learning models to low-level adversarial perturbations and serves as a valuable tool for researchers to develop and evaluate new defense techniques.

The authors thoroughly investigate the transferability of low-level adversarial attacks, which is a critical aspect of their practicality. By studying the success rates of different attack methods and the performance of models under these attacks, the authors gain insights into the vulnerability of models to specific types of perturbations. They also analyze the effect of adversarial training on model robustness, providing a basis for understanding the efficacy of existing defense mechanisms. Through this extensive evaluation, the paper advances the state of knowledge in adversarial machine learning, offering a valuable resource for future research and development in the field.

#### 2.2.1.2 BERT-ATTACK (Li et al., 2020)

Li et al. (2020) presents a notable research contribution that focuses on adversarial attacks specifically targeting BERT models, which are widely used in natural language processing tasks. The study proposes a novel approach for generating adversarial examples by leveraging the power of BERT itself. The authors introduce a two-step process, where they first generate substitute tokens using BERT's masked language model, and then perturb these tokens to craft adversarial examples. By exploiting the linguistic context captured by BERT, the proposed attack demonstrates significant success in deceiving BERT-based models for various natural language processing tasks, including text classification. The work highlights the importance of understanding the vulnerabilities of state-of-the-art models like BERT and provides valuable insights into the potential weaknesses of these models in the face of adversarial attacks. The findings contribute to the ongoing research in adversarial machine learning and call for the development of robust defenses against such attacks to ensure the reliability and trustworthiness of BERT models in practical applications.

#### 2.2.1.3 Word-level Textual Adversarial Attacking as Combinatorial Optimization (Zang et al., 2019)

The authors propose a novel approach to generating adversarial examples at the word-level in natural language processing tasks. The main contribution of this work is the formulation of adversarial attacking as a combinatorial optimization problem, which enables the effective exploration of the search space for high-quality adversarial examples. By leveraging Particle Swarm Optimization (PSO)

and Genetic Algorithm (GA), the authors develop a versatile framework capable of generating imperceptible and semantically valid adversarial examples that can successfully mislead various deep learning models.

The authors' methodology not only provides an innovative perspective on generating word-level adversarial examples but also demonstrates the effectiveness of their approach through extensive experiments on multiple datasets and tasks, such as sentiment analysis and textual entailment. Their results showcase the robustness of the proposed framework in generating adversarial examples that maintain natural language properties, while significantly reducing the victim model's performance. This work serves as an essential milestone for understanding and evaluating the vulnerability of deep learning models to textual adversarial attacks.

#### 2.2.1.4 Generating Natural Language Adversarial Examples (Alzantot et al., 2018a)

This paper presents a novel framework for generating natural language adversarial examples, which are text inputs crafted to mislead machine learning models without significantly changing the meaning of the original text. The authors propose a two-step approach: (1) a genetic algorithm for searching over possible transformations to the input text, and (2) a deep reinforcement learning-based language model to fine-tune the transformations while maintaining the original meaning. The framework is demonstrated to be effective in generating adversarial examples for text classification tasks, causing state-of-the-art models to make incorrect predictions with high confidence. The work highlights the vulnerability of existing NLP models to adversarial attacks and emphasizes the need for more robust models and training mechanisms.

### 2.2.2 Attacks on Arabic Language

#### 2.2.2.1 Adversarial examples in Arabic (Alshemali and Kalita, 2019)

Alshemali and Kalita (2019) introduces a word-level attack strategy that focuses on breaking a simple grammatical rule in Arabic, specifically the noun-adjective agreement. The authors justify their choice by emphasizing that adjectives often

convey the emotional tone of a sentence, making them crucial for sentiment analysis tasks. Termed the "Adjectives Attack" in this paper, the approach targets the most influential token in order to maximize the impact of the modification while preserving the sentence's meaning. The authors evaluate the effectiveness of the Adjectives Attack on two word-level models: the BiLSTM model proposed by Gao et al. (2018b), trained on the HARD corpus, and Kim's word-level CNN model (Kim, 2014), trained on the BRAD corpus (Elnagar and Einea, 2016).

#### 2.2.2.2 Character-level adversarial examples in Arabic (Alshemali and Kalita, 2021)

This paper presents a novel character-level attack designed to exploit the high occurrence of errors resulting from typographical mistakes and spelling errors, especially when considering individuals who are non-native Arabic learners. This attack leverages a lookup table containing groups of visually similar letters, allowing for the flipping of one or two characters in a word. The authors employ this approach to deceive machine learning models trained on Arabic text. The study evaluates the effectiveness of this character-level attack on two different models: the BiLSTM model proposed by Gao et al. (2018b), trained on the HARD corpus, and Kim's word-level CNN model (Kim, 2014), trained on the BRAD corpus (Elnagar and Einea, 2016).

# Chapter 3

# Methodology

## 3.1 Introduction

The methodology chapter serves as a roadmap for our study, outlining the key steps and approaches undertaken to investigate adversarial attacks on machine learning models trained for offensive speech detection in Arabic language. In this chapter, we provide a detailed account and thorough explanation of the adversarial attacks employed in our research. Additionally, we present a comprehensive description of the different algorithms and approaches utilized to apply these adversarial attacks, ensuring a comprehensive understanding of their implementation and impact.

By delving into the methodology, we aim to provide a clear and transparent framework for our study. We explore various types of adversarial attacks including character-level and word-level attacks, each designed to manipulate and deceive machine learning models, with a specific focus on their implications for offensive speech detection. These attacks challenge the robustness of the models and highlight potential vulnerabilities that may exist. Our primary objective was to explore the robustness and vulnerabilities of transformer-based models, specifically Arabert and Marbert, when subjected to adversarial attacks, with a particular focus on the context of the Arabic language.

Through the detailed description of the methodology, we aim to contribute to the broader understanding of adversarial attacks on offensive speech detection models. By shedding light on the implementation details and approaches, we provide a foundation for further research, advancements, and the development of

robust defenses against adversarial attacks. The subsequent sections will provide a comprehensive breakdown of the specific adversarial attacks employed and the algorithms utilized in our investigation.

## 3.2 Naïve Approach for Arabic specific Attacks

In this approach, adversarial attacks are specifically designed to violate specific rules of the Arabic language. These attacks adhere to predefined rules, where each step follows specific criteria. The rules encompass changing the most important token, applying the change only if the token meets certain criteria, and considering the attack unsuccessful if the criteria are not met. The modification applied to the token is determined by the attack type and algorithm.

These attacks are categorized as naive since they restrict the exploration of alternative changes within the sentence, limiting modifications solely to the most crucial token. This places greater emphasis on the method used to measure the importance of each token. Furthermore, the criteria that must be satisfied by the token for the modification to be applied further constrains the attack surface. If the criteria are not met, the attack is deemed unsuccessful, and no modification is made. This limitation can potentially impact the performance of these attacks.

In our study, we applied two adversarial attacks using this approach: a word-level attack known as the adjective attack and a character-level attack called the flip attack. The subsequent sub-sections will provide a comprehensive explanation of these attacks, outlining their methodologies, techniques, and implications

### 3.2.1 Adjective Attack

This attack, as described by (Alshemali and Kalita, 2019), specifically targets the violation of adjective rules in the Arabic language. It capitalizes on the linguistic principle that adjectives and nouns in Arabic must agree in terms of number, gender, and definiteness. Based on this knowledge, the attack operates as follows: given an input sentence, if the most important token is identified as an adjective, the adversarial example is constructed by modifying this token to disrupt the adjective-noun agreement. Conversely, if the most important token is not an adjective, it remains unchanged.

The attack is constructed as follows:

1. **Identifying the most important token:**

   In this step, we employ the R1S scoring function (Gao et al., 2018a) to identify the most important token within the given sentence. However, it is important to note that the most important token may be a part of a larger word, which is not conducive to this attack. To address this, we reconstruct the word containing the most important token and utilize this word as the basis for the attack. This reconstructed word will be referred to as the 'most important word' throughout to prevent any confusion.

2. **Identifying the POS of the selected token:**

   The morphological features resulting from the morphological analyzer can be studied using the POS Tagging functionality provided by CAMeL tools (Obeid et al., 2020a). This functionality enables us to determine whether the most important word is an adjective or not by examining its part-of-speech tag. If the word is identified as an adjective, we proceed to the following step of the attack. If it is not an adjective, we leave it unchanged and repeat step 1 for the subsequent input sequence.

   To ensure the flexibility and inclusivity of our approach across different dialects, the part-of-speech identification process considers the various databases and dialects encompassed within CAMeL tools. Specifically, we compare the results obtained by analyzing the word using the four different dialects available and assign the word to the first dialect that identifies it as an adjective.

3. **Transformation of the adjective:**

   Next, if the most important token is identified as an adjective, we proceed to perform a transformation on the word to disrupt the adjective-noun agreement grammatical rule. This transformation is accomplished using the Reinflection functionality available in CAMeL tools. The Reinflector takes as input the most important word and a list specifying the morphological features with the required modification.

   We introduce four distinct transformations as follows:

   - **Plural:** This transformation focuses on modifying the Number feature. If the word is in the Singular form, it is altered to become Plural, and vice versa.

- **Dual:** Similarly, in this case, the Number feature is modified. If the word is in the Singular form, it is transformed to the Dual form, and vice versa.
- **Gender:** The Gender feature is the target of modification in this transformation. If the word is masculine, it is converted to the feminine form, and vice versa.
- **Definite:** Lastly, the Proclitic Feature is the focus of modification in this transformation. If the word is already defined by a determiner, it is removed. If it is not defined, a determiner (ال) is added.

| Attack Type | Result |
|---|---|
| **Plural** | جيدون |
| **Dual** | جيدان |
| **Gender** | جيدة |
| **Definite** | الجيد |

TABLE 3.1: Example on the Adjective Attack transformations for the word 'جيد'

Table 3.2 provides an example illustrating this attack.

| Attacker | Sentence | Label |
|---|---|---|
| Original | هذا اعدام عيل | Offensive |
| Adjective Attack | هذا الاعدام عيل | Non-Offensive |

TABLE 3.2: Adjective Attack Example

### 3.2.2 Flip Attack

The inspiration for this attack originates from (Alshemali and Kalita, 2021).

The motivation behind this attack stems from the prevalence of human errors in typographical mistakes and spelling errors, particularly among non-native Arabic learners. Spelling mistakes often arise from a misinterpretation of the proper word structure, such as substituting one character for another. Research indicates that non-native Arabic learners are prone to making these mistakes, often by confusing visually similar letters. Due to the inherent visual resemblances within the Arabic alphabet, non-native learners frequently encounter difficulties in distinguishing

| Group | Characters | | | | | |
|---|---|---|---|---|---|---|
| ١ | ب | ت | ث | ن | ي | ئ |
| ٢ | ج | ح | خ | | | |
| ٣ | د | ذ | | | | |
| ٤ | ر | ز | | | | |
| ٥ | س | ش | | | | |
| ٦ | ص | ض | | | | |
| ٧ | ط | ظ | | | | |
| ٨ | ع | غ | | | | |
| ٩ | ق | ف | | | | |
| ١٠ | ك | ل | | | | |
| ١١ | ه | ة | | | | |
| ١٢ | و | ؤ | | | | |
| ١٣ | ي | ى | ئ | | | |
| ١٤ | ت | ث | ب | | | |

TABLE 3.3: Resemblance groups in Arabic Alphabet

between similar-looking letters. Table 3.3 illustrates groups of alphabet characters that are prone to confusion, combining insights from the authors' derivation (Alshemali and Kalita, 2021) and our own refinements.

By referring to Table 3.3, the Candidates are generated through character substitution. Specifically, we have implemented two types of character substitutions, namely Flip1 and Flip2. Flip1 involves the substitution of a single character, while Flip2 involves the substitution of two characters. This attack can be expanded to accommodate a varying number of character substitutions. However, if a token does not contain any of the specified letters, no substitution will occur.

Upon examining Table 3.4, it becomes evident that the groups presented in Table 3.3 are categorized into three distinct groups. These various classifications are taken into consideration in the substitution process.

| Type | Description | Groups |
|---|---|---|
| 1 | characters visually similar regardless of their position in a word | 2,3,4,5,6,7,8,9,10 |
| 2 | characters visually similar at the beginning or the middle of a word | 1 |
| 3 | characters visually similar at the end of a word | 11,12,13,14 |

TABLE 3.4: Types of Resemblance groups in Arabic Alphabet

Subsequently, all meaningful substitutions are disregarded from the derived combinations. This condition is established to address the problem of perceptibility,

as a person is less likely to misperceive the original meaning of a perturbed word if the new word is meaningless. We utilize CAMel Tools (Obeid et al., 2020b) to determine whether the word generated through the attack is meaningful or not.

## 3.3 Scoring Function

The scoring function plays a crucial role in the context of adversarial attacks on NLP models. Its primary objective is to assign importance scores to individual words within a sentence, based on their potential impact on the model's prediction. By ranking the words according to their relevance and influence, the scoring function provides valuable insights that can be leveraged to optimize the adversarial attack process.

The purpose of this ranking is twofold. First, it allows attackers to identify the most influential words in the input sentence, enabling them to focus their perturbation efforts on these key targets. By prioritizing the most impactful words, attackers can strategically craft subtle modifications that achieve maximum effectiveness while minimizing the extent of perturbation. This targeted approach not only increases the chances of a successful attack but also helps maintain the naturalness and fluency of the adversarial example, making it harder for the model to detect the manipulation.

Second, the scoring function aids in understanding the inner workings of the NLP model and its decision-making process. By analyzing the importance scores assigned to different words, researchers and practitioners can gain valuable insights into the model's vulnerabilities and limitations. This knowledge can guide further model improvements, robustness enhancements, and the development of countermeasures against adversarial attacks.

In our context, we focus on the first objective of the scoring function which is identifying the most influential words to be the main focus of the attack.

The scoring function can be implemented in various ways, with one commonly used approach involving the replacement of each word or token in the input sentence with the special token [UNK]. This token, which stands for "unknown," is included in the vocabulary of every NLP model and is used to represent any word that falls outside the scope of the model's predefined vocabulary dictionary. By substituting

out-of-vocabulary words with the [UNK] token, the scoring function captures the model's response and records the corresponding logits. Other approaches, like removing the word, can be used as well.

The rationale behind employing the [UNK] token is that its embedding representation is typically generic and considered insignificant to the model. Since the [UNK] token is used to represent unknown words, its embedding is designed to be neutral and non-discriminatory. This characteristic ensures that the substitution of out-of-vocabulary words with the [UNK] token does not introduce substantial bias or influence into the model's predictions during the scoring process.

It is important to note that while the [UNK] token is a common choice for representing unknown words, different models may employ alternative strategies, such as using a specific token or embedding representation for unknown words. The exact implementation details may vary depending on the specific NLP model and its architecture.

By inferring the model with the modified input, where each word is replaced by [UNK], the scoring function effectively captures the logits associated with the [UNK] token. Analyzing the logits can provide insights into how the model reacts to unfamiliar or out-of-vocabulary words, allowing for the identification of vulnerabilities or potential areas for adversarial attack.

These logits represent the model's prediction or confidence score for the unknown word placeholder and they can be used to calculate the score of this word using various equations which we will discuss in the following subsections.

**Basic Approach** is the Replace-1 scoring (R1S) function introduced in (Gao et al., 2018a).

Using the black-box setting, the R1S function directly indicates the effect of changing the required input token $x_i$ by another token $x'_i$

$$R1S(x_i) = F(x_1, x_2, ..., x_{i-1}, x_i, ..., x_n) - F(x_1, x_2, ..., x_{i-1}, x'_i, ..., x_n)$$

In our context, $x'_i$ is chosen to be [UNK] token. A higher R1S value suggests a higher importance of the token $x_i$.

This method is straightforward, however it does not take into account if the substitution led to misclassification, which is a crucial factor when developing an adversarial attack.

**Example** Consider a 4-class problem.
The original sentence was correctly classified and the original logits were:
$[0.35, 0.2, 0.15, 0.3]$
The logits after replacing the first word became:
$[0.31, 0.21, 0.18, 0.3]$
The logits after replacing the second word became:
$[0.32, 0.2, 0.15, 0.33]$
The R1S score for the first word will be 0.04 which is higher than the score of the second word which is 0.03. However, replacing the second word has caused a misclassification which is usually the main purpose of the adversarial attack.

**Modified Approach** is introduced in (Li et al., 2020). It is not mentioned explicitly in the paper, however, it is implemented in the code published with the paper. It computes the score as follows:

$score = originalClass_{\text{old logit value}} - originalClass_{\text{new logit value}} + (misclassificationOccurs) * (misclassifiedClass_{\text{new logit value}} - misclassifiedClass_{\text{old logit value}})$

The variable $misclassificationOccurs$ takes the value 1 if a misclassification occurs and 0 otherwise. The old logit value refers to the model's logits before replacing the candidate word and the new logit value refers to the model's logits after the replacement.

The term $(misclassifiedClass_{\text{new logit value}} - misclassifiedClass_{\text{old logit value}})$ is added to the score to account for the misclassification.

If we reconsider the last example, we will see that the modified approach gives the first word a score of 0.04 and the second word a value of 0.05, which is higher because it led to a misclassification.

**Our Approach** is founded on the same idea as the prior approach, which accounts for the misclassification in addition to the decline in the model's confidence

| Class 1 | Class 2 | Class 3 | Class 4 | Score |
|---|---|---|---|---|
| 0.1 | 0.2 | 0.6 | 0.1 | -0.3 |
| 0.2 | 0.3 | 0.3 | 0.2 | 0.08 |
| 0.1 | 0.5 | 0.4 | 0.0 | 0.1 |
| 0.0 | 0.6 | 0.4 | 0.0 | 0.2 |
| 0.05 | 0.5 | 0.4 | 0.05 | 0.95 |

TABLE 3.5: Scoring different examples using our scoring function with class 3 as the correct class

in the initial class. We also provide a new criterion that measures how overconfident the model is in the incorrect class. We compute the score as follows:

$$score = \sum_{i=1, i \neq j}^{n} c_i^2 - c_j^2$$

where $c_i$ is the model's score for class $i$ after replacing the candidate word, $j$ is the initial correct class and $n$ is the number of classes.

The scores will range from 1 to -1, with 1 being the most important and -1 being the least important. When the model is overconfident in the wrong class, the score rises. Ideally, the incorrect class will have a value of 1 and the remaining classes will have zero values, resulting in a score of 1. When the model is still confident in the correct class after the replacement, it will receive the lowest score. Ideally, the correct class will have a value of 1 and the other classes will have zero values, resulting in a score of -1.

Table 3.5 illustrates how our function handles different cases.

## 3.4 Introduction to our Approach

Our proposed approach entails dividing the adversarial attack into two distinct stages: candidate generation for each word and the subsequent search for the optimal combination of candidates through the use of a search algorithm. This division enables us to efficiently process replacements on the entire dataset as a batch, rather than having to infer the model multiple times. In the subsequent section on Search Algorithms, we will delve deeper into this methodology. First, however, we will provide a comprehensive analysis of candidate generation using two different approaches: word-level and character-level.

## 3.5 Candidate Generation

Candidate generation involves generating modified versions of original text inputs to create potential adversarial examples. Specifically, in NLP, candidate generation is particularly important for character-level and word-level attacks. These attacks aim to manipulate individual characters or words in the text strategically to deceive ML models. The ultimate objective is to generate modified text that appears natural while intentionally causing misclassifications or misleading predictions. The development of effective candidate generation techniques is essential for assessing model vulnerabilities and building robust defense mechanisms to counter adversarial threats.

### 3.5.1 Word-Level Attacks

There are several methods for generating candidates that are similar to the original word. These include simple tactics like, the Adjective Attack outlined in the Naive Arabic Specific Attacks section, as well as additional ways such as employing word synonyms, language models, and sememes (Zang et al., 2019).

The technique we will focus on is to use a language model to attack itself.

#### 3.5.1.1 BERT Attack

This concept has been introduced in (Li et al., 2020). The attack employs a clever idea in which the language model is used as a tool to attack its fine-tuned versions. The idea behind this approach is that the masked language model (MLM) can generate similar alternatives that can substitute the original word without changing its meaning, but in a different context, this can deceive its fine-tuned version into misclassifying the sentence.

**Algorithm** The BERT Attack algorithm operates as follows: First, the words in the input sentence are ranked using a scoring function, as discussed in the Scoring Functions section. This function assigns a score to each word based on its importance or potential impact on the adversarial attack.

The sentence is then tokenized using the tokenizer of the target model. Each word is converted into its corresponding tokens, taking into account the specific tokenization rules and vocabulary of the model.

The algorithm iterates over each important word in the sentence. If the word remains intact after tokenization, its candidates are filtered, and the algorithm proceeds to the next word. However, if the word is split into subwords during tokenization, the algorithm considers different permutations of these subwords and calculates the perplexity for each permutation using the language model. The permutations with the lowest perplexity are selected as candidates for further evaluation. It is worth mentioning that, due to memory limitations, instead of calculating perplexity for all combinations, the list is limited to a fixed size in the implementation of the paper. It takes at most 12 subwords and their top 4 candidates and only uses the first 24 combinations to get the perplexity.

The algorithm then evaluates the candidates and replaces the original words using a greedy approach which we will discuss in a subsequent section.

By following this iterative process of candidate generation, evaluation, and replacement, the BERT Attack algorithm aims to find the most effective sequence of word replacements that leads to misclassification while minimizing the reduction in the model's confidence for each substitution.

The previous steps can be illustrated in 3.1

**Filtering** During the candidate filtering process, several measures are implemented. Firstly, any stop words are excluded from consideration. Additionally, a minimum threshold is applied to the scores assigned to candidates by the Masked Language Model (MLM). This threshold ensures that only candidates with sufficiently high MLM scores are retained.

Moreover, a minimum threshold is enforced on the cosine similarity between the embedding vector of the current word and the embedding vectors of its candidates. This criterion is implemented to prevent the replacement of a word with its antonym. This precaution is taken because the MLM might not consistently differentiate between antonyms when they frequently co-occur in similar contexts.

In the case of subwords, a maximum threshold is applied to their perplexity scores. This threshold serves to control the inclusion of subword permutations in the

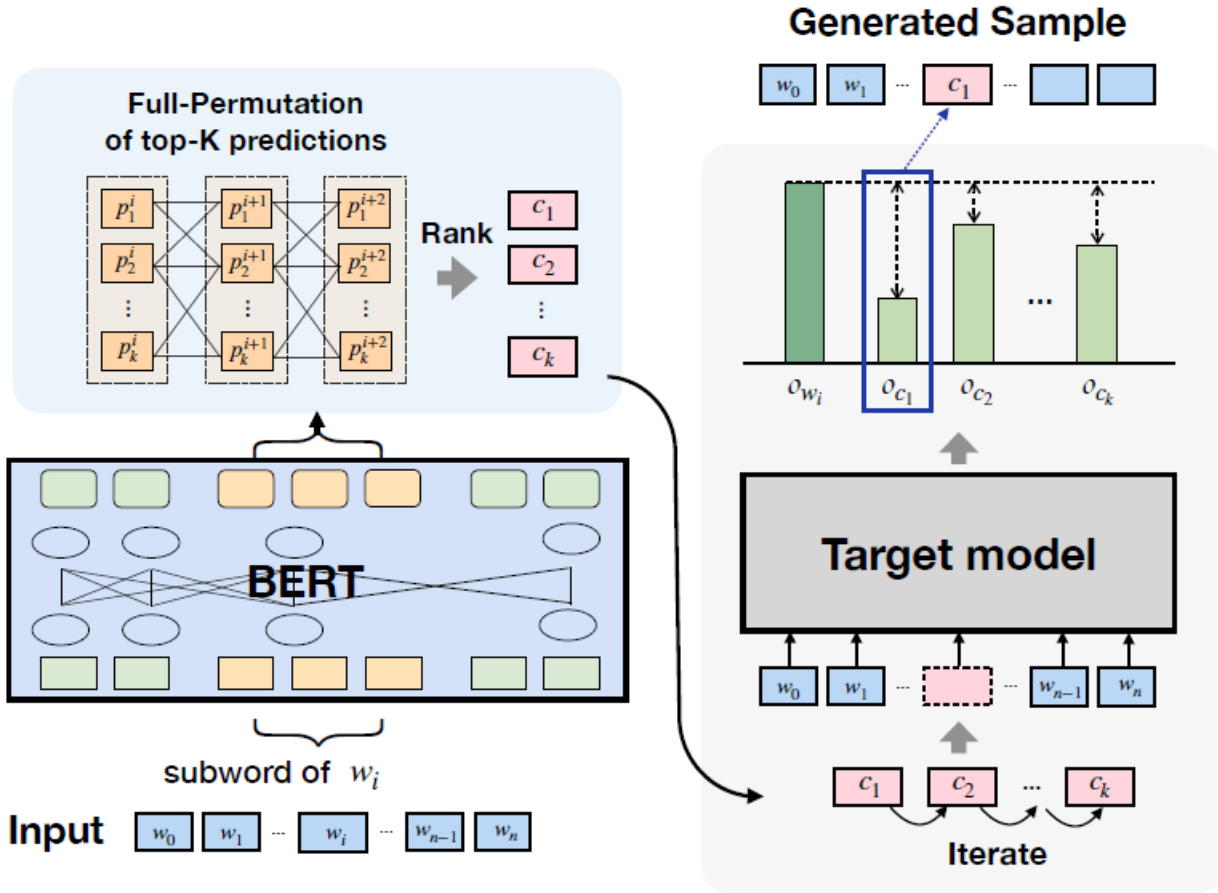



FIGURE 3.1: Illustration of BERT attack replacement process

candidate generation process. By setting an upper limit on perplexity scores, the algorithm ensures that only subword permutations that exhibit a reasonable level of coherence are considered as candidates.

By incorporating these filtering mechanisms, the algorithm endeavors to maintain semantic consistency, contextual relevance, and sensible replacements, thereby enhancing the quality and appropriateness of the generated candidates for the adversarial attack.

This method is intended to target a single English sentence using the BERT paradigm. We have to tweak a few details to adapt this to the Arabic Domain and optimize this technique to run efficiently on the entire dataset. We have also implemented extra filtering stages unique to the hate speech detection task to ensure the attack's validity.

**Modification specific to the Arabic Domain** To adapt this to the Arabic Domain, we have used two models to generate the candidates: AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020) models.

Because of the nature of the Arabic language, subjects, and identification letters are attached to the word as a prefix or a suffix. If the language model leverages Farasa, as in the case of the AraBERT model, this can be addressed. This is why we have added the filtering process using the stop words (Taher, 2023) on all candidates including intact words and subwords.

**Modification specific to the Hate Speech Task** To ensure the validity of the attack, we had to ensure that it did not remove the slur from the offensive sentence or add a slur to the non-offensive sentence. We utilized the toxicity list Toxicity-20 (Costa-jussà et al., 2022) to implement this. We have imposed a filter that states that if a term is on this list, it can only be replaced by a contender on this list. Furthermore, if a term does not appear on the list, it cannot be replaced by a word from the list.

To ensure that the sacred name of Allah is not inappropriately inserted into contexts where it is not suitable, we have included the word الله in our list of stop words. By designating it as a stop word, our algorithm avoids generating or inserting this word in contexts that may not be appropriate or respectful to the reverence associated with the name of Allah. This consideration aligns with our commitment to maintaining the sanctity and appropriate usage of religious terms throughout the adversarial attack process.

**Optimization Tricks** As we are interested in running the attack on the entire data, we had to put utilize the parallelism of the GPU and batch the inference data to the model. Instead of inferring one sentence at a time, we initially utilized the MLM model to create all candidates for all tokens for each sentence in the data as a batch. Then, on all candidates, we applied the score threshold filter, the cosine similarity threshold filter, the stop words filter, and the toxicity filter for the untokenized words.

As for the perplexity calculations, to avoid loading the model for each word, we employed batching to calculate the perplexity scores for all subwords in the data. Instead of limiting the subword permutation list to a set length, we used a dynamic

one. We choose to take no more than 8 subwords and ignore words that are divided into more than 8 tokens. We then decided to select k candidates for each subword. As the number of subwords increases, the value of k decreases. We have used a simple equation $k = K_{\max} - x$ where $K_{\max}$ is the maximum number of candidates taken and $x$ is the number of subwords. It is worth mentioning that we only counted the subwords that can be potentially replaced.

After obtaining the potential candidates for the tokenized words, we apply the stop words filter and the toxicity filter to them.

We have all feasible filtered candidates for each word in each sentence in the dataset at the end of these processes. The next stage will be to figure out how to efficiently use this list to select the optimum perturbation.

Table 3.6 provides an example illustrating this attack.

| **Attacker** | **Sentence** | **Label** |
|---|---|---|
| Original | هذا اعدام عيل | Offensive |
| BERT Attack | هذا موت عيل | Non-Offensive |

Table 3.6: BERT Attack Example

### 3.5.2 Character-Level Attacks

In this subsection, we will explore the process of candidate generation in various character-level attacks, including space insertion, the addition of special characters, letter separation, one-letter English flip, and inner shuffle.

#### 3.5.2.1 Space Insertion Attacks

The inspiration for this attack originates from (Eger and Benz, 2020), which highlights that this particular type of attack is commonly observed in social media platforms.

The motivation behind this attack is to gain a deeper understanding of how the model behaves and responds when white spaces are inserted within the text. By introducing additional spaces, the original text is segmented into multiple tokens, which may lead to a misinterpretation of the desired meaning by the model. This

attack specifically targets the model's sensitivity to spacing variations and its ability to correctly comprehend the intended message.

The rationale behind this attack lies in the inherent differences between machine interpretation and human understanding. While humans can easily grasp the intended meaning of a text despite spacing manipulations, machine learning models may struggle to accurately interpret the same text when spaces are altered.

This attack can be executed in two different ways. In the first approach, spaces are inserted in the middle of the text, causing the token to be split into two distinct segments. The objective is to observe how the model's predictions or interpretations may be affected by this single space insertion. In the second approach, spaces are inserted at various positions within the text, generating multiple candidate variations.

#### 3.5.2.2 Addition of Special Characters Attacks

The inspiration for this attack originates from (Eger and Benz, 2020), which highlights that this particular type of attack is commonly observed in social media platforms same as Space Insertion Attacks.

The motivation behind this attack is similar to that of Space Insertion Attacks 3.5.2.1, with a slight modification in the approach. Rather than splitting the token into two separate tokens, this attack aims to keep the token as a single entity while introducing a special character within it. The goal is to examine how the model perceives and interprets the token, which may appear different to the model compared to human understanding. While humans can easily recognize the original word within the token despite the presence of the special character, the model may struggle to accurately identify it. This discrepancy in perception highlights the importance of investigating the model's response to such variations and developing strategies to improve its ability to handle tokens with embedded special characters.

This attack can be executed in two different ways. In the first approach, a special character is inserted in the middle of the text, causing the token to have a special character in it. The objective is to observe how the model's predictions or interpretations may be affected by this single special character insertion. In the

second approach, special characters are inserted at various positions within the text, generating multiple candidate variations.

#### 3.5.2.3 Letters Separation Attacks

The motivation behind this attack was to investigate the model's response when the token is divided into individual letters. This attack can be executed in two distinct ways. The first approach involves separating all the letters with spaces, causing the model to deal with each character as a separate token. This makes it more challenging for the model to discern the intended meaning of the token. However, this attack also poses some difficulty for humans, as it requires greater concentration to reconstruct the word from the separated letters. In the second attack, all the letters are separated by special characters. In this scenario, the model encounters a single token with special characters interspersed between each pair of letters. Remarkably, humans can easily know the original word despite the presence of these special characters.

In the first type of attack, candidates are generated by inserting spaces between all the letters, while in the second type, candidates are generated by incorporating any of the special characters between the letters.

#### 3.5.2.4 One-Letter English Flip Attack

The inspiration for this attack originates from social media platforms, where it is commonly used, particularly in regions such as Egypt, where a significant number of individuals use Franco-Arabic (Franko) instead of Arabic or English.

The motivation behind this attack was to investigate the model's response to a specific type of attack, wherein a single letter within a token is altered. The objective was to examine how the model would handle this scenario, while humans, on the other hand, can easily recognize the word by mentally translating the modified letter back to its original form.

The candidates generated in this attack include all the possible Arabic transliterations mentioned in Table 3.7. We also considered specific cases mentioned in Table 3.8, taking into account the inclusion of the "_" character to ensure visual correctness and facilitate the reconstruction of the original word.

| Arabic | Transliteration |
|---|---|
| أ | a |
| ا | a |
| إ | e |
| ب | b |
| ت | t |
| ث | th |
| ج | g |
| ح | h |
| خ | kh |
| د | d |
| ذ | z |
| ر | r |
| ز | z |
| س | s |
| ش | sh |
| ص | s |
| ض | d |
| ط | t |
| ع | 3 |
| غ | gh |
| ف | f |
| ق | q |
| ك | k |
| ل | l |
| م | m |
| ن | n |
| ه | h |
| و | w |
| ي | y |

TABLE 3.7: Arabic Transliterations

| Special Cases |
|---|
| و ، ز ، ر ، ذ ، د ، إ ، ا ، أ |

TABLE 3.8: Special Cases in Arabic Transliterations

#### 3.5.2.5 Inner Shuffle Attack

The inspiration for this attack originates from (Eger and Benz, 2020)

The motivation behind this attack is the human ability to understand words even when all the letters are shuffled except the first and last letters. However, ML models may struggle to accurately know the original word in such cases.

The candidates generated in this attack are all the possible shuffles of the middle letters. However, if the length of the word is smaller than 4 letters, no candidates are generated.

Table 3.9 provides an example illustrating all character-level attacks.

| **Attack** | **Sentence** | **Label** |
|---|---|---|
| Original | هذا اعدام عيل | Offensive |
| Flip Attack | هذا اعذام عيل | Non-Offensive |
| Space Insertion Attack (Middle of the word) | هذا اعــ دام عيل | Non-Offensive |
| Space Insertion Attack (Anywhere in the word) | هذا ا عدام عيل | Non-Offensive |
| Special Characters Attack (Middle of the word) | هذا اعــ!دام عيل | Non-Offensive |
| Special Characters Attack (Anywhere in the word) | هذا ا&عدام عيل | Non-Offensive |
| Letters Separation Attack (Spaces) | هذا ا عــ د ا م عيل | Non-Offensive |
| Letters Separation Attack (Special Characters) | هذا ا!ع!د!ا!م عيل | Non-Offensive |
| One-Letter English Flip Attack | هذا اعدا m عيل | Non-Offensive |
| Inner Shuffle Attack | هذا اعادم عيل | Non-Offensive |

Table 3.9: Character-level Attacks Example

## 3.6 Search Algorithms

Search algorithms for adversarial examples in the context of word substitution with given candidates primarily aim to generate text inputs that can deceive natural language processing models while maintaining semantic coherence and readability. These algorithms typically involve exploring the search space of possible word substitutions, guided by domain-specific knowledge and heuristics.

They may employ methods such as greedy search and optimization techniques, as we will explore briefly, or genetic algorithms as mentioned before in (Alzantot et al., 2018a) to evaluate and select candidate word replacements based on their impact on the model's output and their adherence to the original input's meaning. The effectiveness of these search algorithms is essential for crafting adversarial examples that can expose vulnerabilities in NLP systems and help improve their robustness against malicious attacks.

This is a combinatorial optimization problem aiming to find the combination of word replacements that fool machine learning models while preserving meaning. We explored some of the approaches and techniques to be used to produce an adversarial example from the possible search space.

### 3.6.1 Greedy Algorithm

The algorithm was adopted in BERT attack (Li et al., 2020). It proceeds to evaluate each candidate by replacing the current word with the candidate and assessing its impact on the model's classification. If the candidate causes a misclassification, indicating a successful adversarial attack, the algorithm terminates. However, if the candidate does not cause a misclassification, the algorithm determines which candidate results in the least reduction in the model's confidence. This candidate is selected as the replacement for the current word, and the algorithm moves on to the next important word in the sentence. The words are explored in descending order of their importance score calculated by the chosen scoring function.

We can employ a restriction for the permissible changes to the sentence to maintain the attack valid while lowering the perturbation level. For instance, if the change limit is 0.4, we cannot edit more than 40% of the statement. In the Results section, we will give a more extensive study that investigates the relationship between the limit value and the success of the attack.

This algorithm was designed to perform the attack on a single sentence. Before going on to the next word, each candidate of the current word was evaluated separately and then replaced. However, executing this on the entire dataset is computationally demanding. We must load the model an average of $changeLimit * averageWordsNumber * averageCandidatesNumber * sentencesNumber$ times and we have not utilized the GPU's parallelism.

To avoid this costly process, we have pre-prepared the candidates for all words. Then we group all of the candidates for all of the current words in all sentences. Then we check each sentence to see if it was misclassified. If so, we delete it from the following iteration. If it was not, we chose the candidate who had the greatest impact on the model's confidence. Then we go over the remaining sentences again and again until either all choices have been exhausted or the sentences have been misclassified.

The advantage of the greedy algorithm is its simplicity. However, it doesn't consider all possible outcomes of perturbation. One of the disregarded combinations can achieve misclassification with a lower perturbation percentage than the outcome of the greedy approach.

### 3.6.2 Particle Swarm Optimization

Particle Swarm Optimization (PSO) is an optimization algorithm inspired by social behaviors observed in nature, such as bird flocking and fish schooling. It was first introduced by (Eberhart and Kennedy, 1995). PSO is recognized as an effective metaheuristic approach to solving a wide range of optimization problems, including image classification (Omran et al., 2005), part-of-speech tagging (Silva et al., 2012), and text clustering (Cagnina et al., 2008). Empirical studies have also shown that PSO can outperform other optimization algorithms, such as the Genetic Algorithm (Hassan et al., 2005).

PSO is a population-based optimization technique that utilizes a group of interacting agents, or particles, to iteratively search for an optimal solution within a given search space. Each particle has a position in the search space and moves with an adaptable velocity. In the following sections, the PSO algorithm will be described in detail, along with its mathematical formulation.

#### 3.6.2.1 Particle Swarm Optimization Algorithm

**Problem Representation** Consider an optimization problem in a D-dimensional continuous search space $S \subset \mathbb{R}^D$. A swarm contains N particles, where the position and velocity of each particle can be represented by $x_n \in S$ and $v_n \in \mathbb{R}^D$, respectively, for $n \in \{1, \ldots, N\}$.

1. **Initialization** At the beginning of the algorithm, each particle's position $x_n$ is randomly initialized within the search space, and its velocity $v_n$ is assigned an initial value for each dimension $d \in \{1, \ldots, D\}$ such that $v_{n,d} \in [-V_{\max}, V_{\max}]$.

2. **Evaluation and Recording** For each position in the search space, an optimization score is obtained based on an objective function. The highest optimization score a particle has reached is recorded as its individual best position, denoted as $p_n$. The best position among all individual best positions in the swarm is recorded as the global best position, denoted as $p_g$.

3. **Termination Criterion** If the global best position has achieved a predefined or desired optimization score, the algorithm terminates and outputs the global best position as the final solution.

4. **Velocity and Position Update** If the termination criterion is not met, the velocity and position of each particle are updated based on the particle's current position, individual best position, and the global best position. The update equations are as follows:

$$v_{n,d} = \omega v_{n,d} + c_1 r_1 (p_{n,d} - x_{n,d}) + c_2 r_2 (p_{g,d} - x_{n,d}),$$

$$x_{n,d} = x_{n,d} + v_{n,d},$$

where $\omega$ is the inertia weight, $p_{n,d}$ and $p_{g,d}$ are the d-th dimensions of the n-th particle's individual best position and the global best position, respectively, $c_1$ and $c_2$ are acceleration coefficients that control how fast the particle moves towards its individual best position and the global best position, and $r_1$ and $r_2$ are random coefficients.

After updating the velocities and positions, the algorithm returns to the Evaluation and Recording step, and the process is repeated until the termination criterion is satisfied.

#### 3.6.2.2 PSO-based Adversarial Search Algorithm

Particle Swarm Optimization (PSO) has been applied to the search for adversarial examples in various domains. In this section, we present a modified PSO algorithm designed for word-level adversarial example search. The search space is discrete, with each position corresponding to a sentence, and each dimension of a position representing a word.

**Problem Representation** Consider a D-dimensional discrete search space where each position represents a sentence (or an adversarial example). Each dimension of a position corresponds to a word, represented as:

$$x_n = w_{n1} \dots w_{nd} \dots w_{nD}, \quad w_{nd} \in V(w_d^o),$$

where $D$ is the length (word count) of the original input, $w_d^o$ is the $d$-th word in the original input, and $V(w_d^o)$ is composed of $w_d^o$ and its substitutes.

The optimization score of a position is the target label's prediction probability given by the victim model. The target label is the desired classification result for an adversarial attack.

In this modified PSO, a particle's velocity is related to the position change probability, meaning $v_{nd}$ determines how probable $w_{nd}$ is to be substituted by another word.

### Algorithm Steps

1. **Initialization** Instead of random initialization, the initial position of a particle is determined by randomly substituting one word of the original input. This operation is similar to the mutation in a genetic algorithm (Higashi and Iba, 2003). The positions of $N$ particles are initialized by repeating this mutation $N$ times. Each dimension of each particle's velocity is randomly initialized between $-V_{\max}$ and $V_{\max}$.

2. **Evaluation and Recording**
   This step remains the same as the original PSO algorithm.

3. **Termination Criterion**
   The termination condition is when the victim model predicts the target label for any of the current adversarial examples.

4. **Velocity Update**
   Considering the discrete search space, the original velocity update formula is adapted as follows:

$$v_{nd} = \omega v_{nd} + (1 - \omega)[I(p_{nd}; x_{nd}) + I(p_{gd}; x_{nd})],$$

   where $\omega$ is the inertia weight, and $I(a; b)$ is defined as:

$$I(a; b) = \begin{cases} 1, & \text{if } a = b, \\ -1, & \text{if } a \neq b. \end{cases}$$

   The inertia weight is decreased with the increase of iteration numbers, as suggested by (Shi and Eberhart, 1998):

$$\omega = (\omega_{\max} - \omega_{\min})\frac{T-t}{T} + \omega_{\min},$$

where $0 < \omega_{\min} < \omega_{\max} < 1$, and $T$ and $t$ are the maximum and current numbers of iteration times, respectively.

5. **Position Update**

   The position update is adjusted for the discrete search space using a probabilistic method inspired by (Kennedy and Eberhart, 1997). A two-step position updating process is introduced, in which a new movement probability, $P_i$, is used for a particle to determine whether it moves to its individual best position as a whole.

   The change in each dimension of the particle's position depends on the same dimension of its velocity, specifically with the probability of sigmoid($v_{nd}$). No matter whether a particle has moved toward its individual best position or not, it proceeds to the second step. Here, each particle determines whether to move to the global best position with another movement probability, $P_g$.

   The change in each position dimension also relies on sigmoid($v_{nd}$). $P_i$ and $P_g$ vary with iteration to enhance search efficiency by adjusting the balance between local and global search:

$$P_i = P_{\max} - \frac{t}{T}(P_{\max} - P_{\min}),$$

$$P_g = P_{\min} + \frac{t}{T}(P_{\max} - P_{\min}),$$

   where $0 < P_{\min} < P_{\max} < 1$.

   After the update step, a mutation is applied to each particle to enhance the search in unexplored space. The mutation is conducted with the probability:

$$P_m(x_n) = \min\left(0, 1 - \frac{k}{E(x_n, x_o)}\frac{D}{D}\right),$$

   where $k$ is a positive constant, $x_o$ represents the original input, and $E$ measures the word-level edit distance (number of different words between two sentences). $E(x_n, x_o)/D$ is defined as the modification rate of an adversarial example. After mutation, the algorithm returns to the Evaluation and Recording step.

#### 3.6.2.3 The Performance of PSO

Particle Swarm Optimization (PSO) provides significant benefits over other search approaches like greedy search and genetic algorithms for generating adversarial examples. This compares PSO to some search approaches based on key factors like imperceptibility, validity, naturalness, optimal performance, and complexity.

**Optimization Performance and Balancing Exploration-Exploitation** PSO excels at global optimization, exploring a wider solution space to find more impactful adversarial examples. In contrast, greedy search gets stuck in local optima and genetic algorithms rely on less-effective operators. PSO effectively balances exploration and exploitation by adjusting particle velocities based on individual and global best positions. Greedy search primarily exploits local optima, while genetic algorithms struggle with this balance.

**Simplicity, Flexibility, and Scalability** PSO is straightforward to implement and adapt to various search spaces, including those for word-level adversarial examples. Greedy search requires complex heuristics and genetic algorithms need problem-specific operators. PSO is a scalable, parallelizable, population-based algorithm suitable for large, high-dimensional problems. Greedy search is sequential, genetic algorithms' scalability depends on parameters and operators.

**Perceptibility, Validity, and Naturalness** PSO can find imperceptible, valid, and natural adversarial examples by exploring a wider solution space. Greedy search's local focus and genetic algorithms may limit example quality without careful design.

**Convergence, Parameter Tuning, and Complexity** PSO may converge slower than greedy search, quickly finding local optima. But PSO likely finds higher quality solutions; genetic algorithms' convergence depends on their design. PSO requires some parameter tuning, though less than genetic algorithms. Greedy search needs the fewest parameters but perhaps problem-specific heuristics. PSO may be more complex than greedy search in some cases, though simpler than genetic algorithms.

PSO offers a balanced, flexible, and scalable approach for generating adversarial examples, compared to greedy search and genetic algorithms, for example. PSO's global optimization, simplicity, and ability to find perceptually valid, natural, and imperceptible examples make it well-suited for complex search spaces. While PSO has some disadvantages relating to convergence and parameter tuning, the quality of its results often justifies them.

PSO provides significant benefits for generating adversarial examples but also has some drawbacks compared to alternative approaches like greedy search and genetic algorithms. By leveraging its strengths while being aware of its weaknesses, PSO can be a powerful tool for solving complex optimization problems, such as discovering adversarial examples.

# Chapter 4

# Experimental Setup and Implementation Details

In this Chapter, we present the experimental setup and implementation details of our study. We delve into the dataset employed in our study, outlining its characteristics, source, and any preprocessing steps undertaken. Subsequently, we provide a comprehensive description of the transformer-based models used in our research, including Arabert (Antoun et al., 2020) and Marbert (Abdul-Mageed et al., 2020). We explain the architectures, parameters, and training processes of these models. By detailing these aspects, we aim to provide a clear understanding of the foundations of our experimental setup.

## 4.1 Dataset

The dataset used to train the models for the offensive speech detection task is the *Emojis as Anchors to Detect Arabic Offensive Language and Hate Speech* (Mubarak et al., 2022), which was provided for the OSACT5 (OSACT5, 2022) Arabic hate speech detection tasks.

### 4.1.1 Overview

The dataset consists of annotated tweets that were gathered by specifically targeting emojis commonly associated with offensive communications. These emojis were extracted from two existing datasets of offensive tweets: (Zampieri et al.,

2020) and (Chowdhury et al., 2020). It has been demonstrated that employing emojis as indicators is more effective than relying solely on keywords or patterns. The tweets were extracted from a pool of 4.4 million Arabic tweets collected between June 2016 and November 2017, ensuring that each tweet contained one or more emojis from a predefined list.

To create the labeled dataset for offensive tweet classification a crowdsourcing annotation job was conducted on the Appen platform. Annotators from multiple Arab countries participated, and quality was ensured through 200 manually labeled gold standard questions. Each tweet was assessed by three annotators, with over 190 contributors, considering the subjective nature of judging tweet offensiveness. To further enhance quality, an expert linguist familiar with different dialects was hired and trained to carefully review tweets with conflicting opinions (Mubarak et al., 2022).

### 4.1.2 Label Statistics and Data Partitioning

After eliminating duplicates, near duplicates, and very short tweets, the dataset was reduced to 12,698 unique tweets. Following the annotation process, each tweet was assigned specific labels for the three subtasks as follows:

1. For subtask A (offensiveness detection), each tweet was labeled as either OFF (offensive) or NOT_OFF (not offensive).
2. For subtask B (hate speech detection), the labels HS (hate speech) or NOT_HS (not hate speech) were assigned to each tweet.
3. For subtask C (hate speech type), in addition to the NOT_HS label, tweets were further categorized into HS1 (Race), HS2 (Religion), HS3 (Ideology), HS4 (Disability), HS5 (Social Class), or HS6 (Gender), based on the specific type of hate speech.

However, for our analysis, we focused solely on the labels assigned for subtask A (offensive/not-offensive).

The dataset was divided into three distinct sets: a training set comprising 70% of the data, a development set comprising 10%, and a test set comprising 20%. This consistent split was applied to all subtasks. To provide a comprehensive overview

of the dataset distribution, the following table presents the number of samples associated with each class label within each set:

| Label | Training | Development | Test | Total |
|---|---|---|---|---|
| **OFF** | 3172 | 404 | 887 | 4463 |
| **NOT_OFF** | 5715 | 866 | 1654 | 8235 |

TABLE 4.1: Representation of the data split and number of each label in the OSACT5 Dataset.

### 4.1.3 Data Cleaning

To enhance the quality and consistency of the data before proceeding with the model training, we executed a comprehensive data cleaning process. This involved employing the following cleaning techniques:

1. Replacing Links with 'URL'
2. Removing Twitter-specific symbols including RT (retweet) and <LF>
3. Removing underscore symbol (_)

## 4.2 Models

In this section, we provide an in-depth exploration of the training process employed for the transformer-based models, Arabert and Marbert, specifically tailored for the offensive speech detection task. We discuss the key parameters utilized during the training procedure. Furthermore, we present the results obtained from the training process, providing insights into the effectiveness and capabilities of the trained models in detecting hate speech

### 4.2.1 Parameters

The parameters play a crucial role in shaping the learning dynamics and overall performance of the models. By carefully selecting and fine-tuning these parameters, our aim was to optimize the training process and enhance the models' accuracy in identifying offensive speech.
The table below presents the values assigned to the key parameters utilized during the training process of the transformer-based models, Arabert and Marbert, for offensive speech detection. These parameters include the number of training epochs, initial learning rate, batch size, optimizer, loss function, scheduler, and label smoothing factor.

| **Parameter** | **AraBERT** | **MarBERT** |
|---|---|---|
| **Number of Epochs** | 10 | 10 |
| **Initial Learning Rate** | $10^{-5}$ | $10^{-6}$ |
| **Batch Size** | 16 | 16 |
| **Optimizer** | AdamW | AdamW |
| **Loss Function** | Cross Entropy Loss | Cross Entropy Loss |
| **Scheduler** | Linear Schedule with Warmup | Linear Schedule with Warmup |
| **Label Smoothing** | 0.1 | 0 |

TABLE 4.2: Parameters used for training AraBERT and MarBERT Models.

### 4.2.2 Training Process Results

After meticulous fine-tuning of the parameters to optimize the models' performance for our specific task, in table 4.3 we showcase the best validation accuracy obtained. Alongside the validation accuracy, we also present the corresponding training accuracy. Furthermore, we report the accuracy of the models on the dedicated test set, providing a comprehensive evaluation of their effectiveness. By presenting these accuracy metrics, we aim to showcase the performance achieved by the models and their reliability in identifying offensive speech.

| | **Train** | **Development** | **Test** |
|---|---|---|---|
| **AraBERT** | 86.81% | 85.35% | 83.71% |
| **MarBERT** | 88.26% | 87.17% | 85.83% |

TABLE 4.3: AraBERT and MarBERT training process results.

### 4.2.3 Plots

To provide a comprehensive representation of the training process, this section includes two types of plots for each model. The first plot is the accuracy-epoch plot, which illustrates the model's accuracy on the vertical axis and the corresponding epoch number on the horizontal axis. The plot showcases the accuracy on both the training set (represented by the red curve) and the validation set (represented by the blue curve) after each epoch. This visualization allows us to observe the model's progress and generalization capability throughout the training process.

Similarly, the second plot is the loss-epoch plot, depicting the loss after each epoch for both the training set and the validation set. The loss values on the vertical axis are plotted against the corresponding epoch numbers on the horizontal axis. By examining the loss trends on the training and validation sets, we gain insights into the convergence and optimization of the models.

These plots serve as valuable tools to analyze the training process, visualize the model's performance, and identify any signs of overfitting or underfitting. By presenting these visualizations, we aim to provide a comprehensive understanding of the training dynamics and performance of the models for offensive speech detection.

#### 4.2.3.1 Arabert

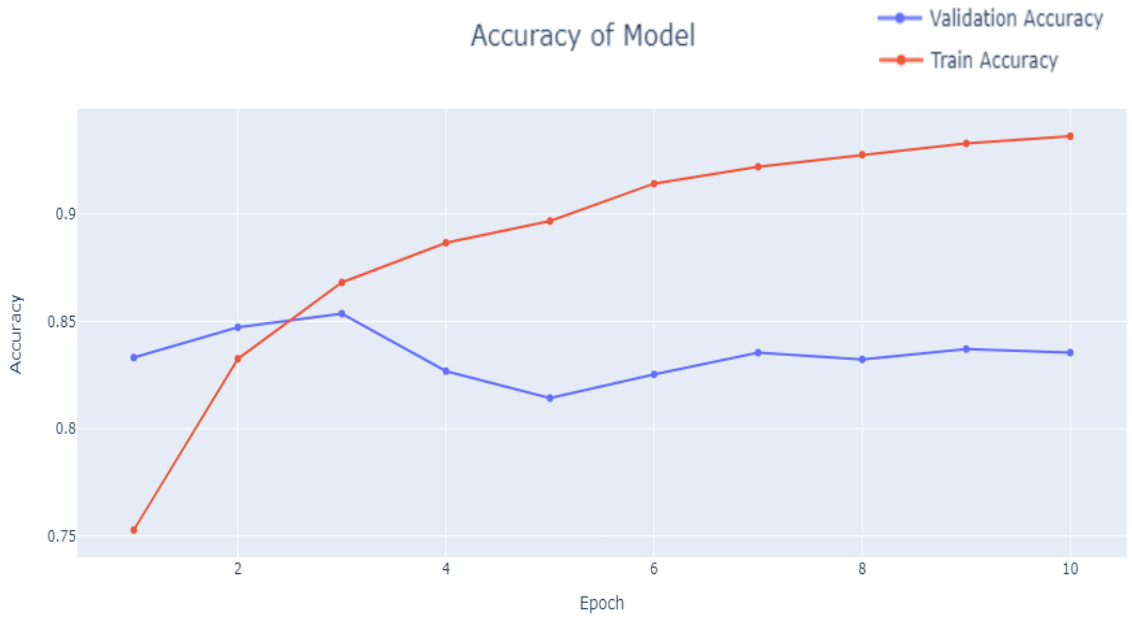



FIGURE 4.1: Accuracy-Epoch Plot for AraBERT Model Training.

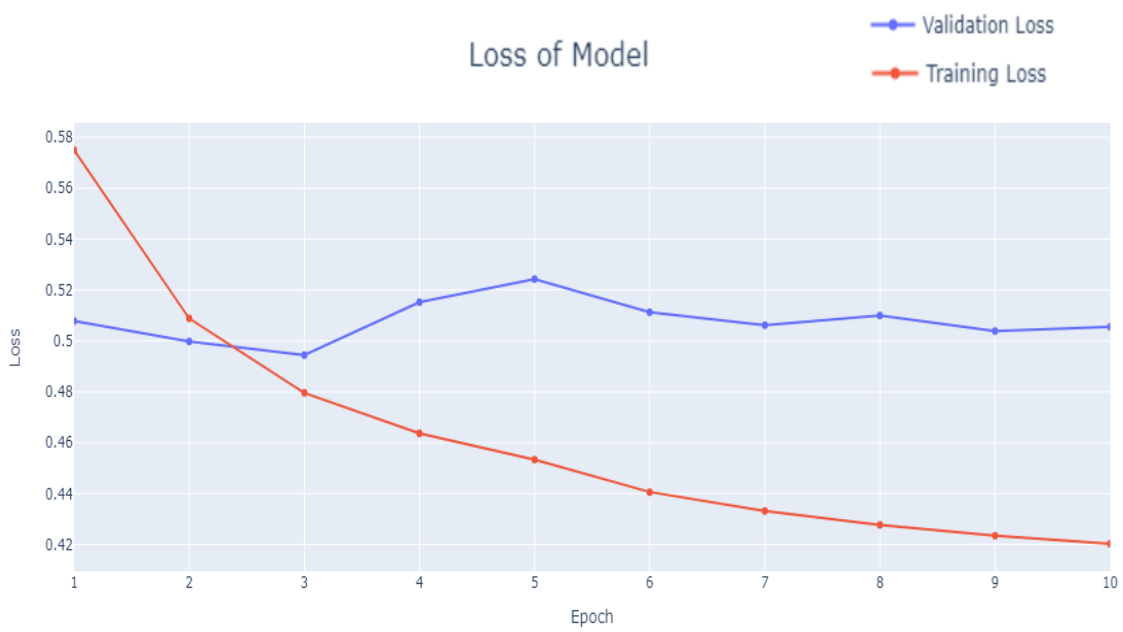



FIGURE 4.2: Loss-Epoch Plot for AraBERT Model Training.

#### 4.2.3.2 Marbert

FIGURE 4.3: Accuracy-Epoch Plot for MarBERT Model Training.

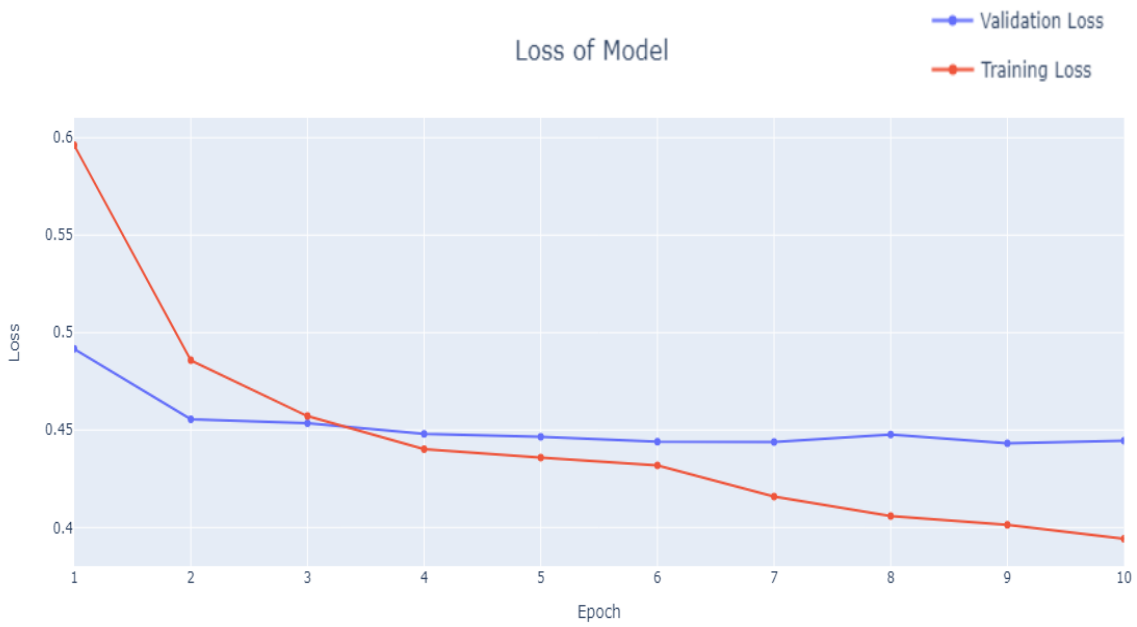



FIGURE 4.4: Loss-Epoch Plot for MarBERT Model Training.

# Chapter 5

# Experiments and Results

In this chapter, we will go over the specifics and outcomes of our studies. First, a comparison between our approach and naive approach on Arabic Specific attacks: Flip Attack and Adjective Attack will be discussed. Then, the hyperparameters for the MarBERT and AraBERT attacks will be presented. Third, we will report the results of running all 12 of our attacks at various change limits on both fine-tuned models: AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020), using offensive sentences from OSACT5 (Mubarak et al., 2022) dataset. Finally, we will discuss more detailed findings based on executing all of our 12 attacks on all of the OSACT5 data (offensive and non-offensive sentences) at a fixed change limit on the AraBERT model.

All of our tests are carried out utilizing the greedy search technique described in the Methodology chapter. The data used in the subsequent trials are previously correctly classified sentences by the target model, implying that the model's accuracy on the data before the attack is 100% on the data used for the attack.

## 5.1 Naive Approach vs. Our Approach

We tested the Adjective and Flip attacks in their simplest forms, as provided in (Alshemali and Kalita, 2019) and (Alshemali and Kalita, 2021). We also ran an additional experiment in which we adjusted the attacks to use our new scoring function and new approach, which divides the process into two stages: Candidate Generation and Search Algorithm.

In the adjective attack, we merged every possible combination of modifications (plural, dual, definitness, and gender) into a list of candidates, and the most effective candidate was chosen using the greedy search algorithm. Only the most important word was attacked in the paper's (Alshemali and Kalita, 2019) methodology, and if it was not an adjective, no attack was carried out. We adjusted our technique and ordered the words based on our scoring function, emptying the candidate list if the word was not an adjective. If the most important word is not an adjective, the algorithm will skip it and move on to the next word. A comparison between the basic approach and our approach results is shown in table 5.1.

| Attack | Offensive Sentences Accuracy |
|---|---|
| Definite Adjective | 98.29% |
| Dual Adjective | 98.46% |
| Plural Adjective | 98.80% |
| Gender Adjective | 98.46% |
| Modified Adjective (limit = 0.4) | **86.66%** |
| Modified Adjective (limit=1) | **86.66%** |

TABLE 5.1: Comparison between basic Adjective Attacks (Definite, Dual, Plural and Gender) and our modified approach at different change limits 0.4 and 1 on AraBERT using previously correctly classified offensive sentences (model's accuracy is 100% prior to the attack)

In the paper (Alshemali and Kalita, 2021), two versions of the Flip attack were introduced: Ar-Flip1 (flipping one character) and Ar-Flip2 (flipping two characters). We've combined both techniques into one, called the Flip Attack. We also used all potential Ar-Flip1 and Ar-Flip2 combinations as a list of candidates and used the greedy search algorithm. A comparison between the basic approach and our approach on the AraBERT model using the correctly classified offensive sentences from OSACT5 is shown in table 5.2.

| Attack | Offensive Sentences Accuracy |
|---|---|
| Ar-Flip1 | 35.77% |
| Ar-Flip2 | 46.50% |
| Flip Attack (limit = 0.4) | 9.74% |
| Flip Attack (limit = 1) | **1.2%** |

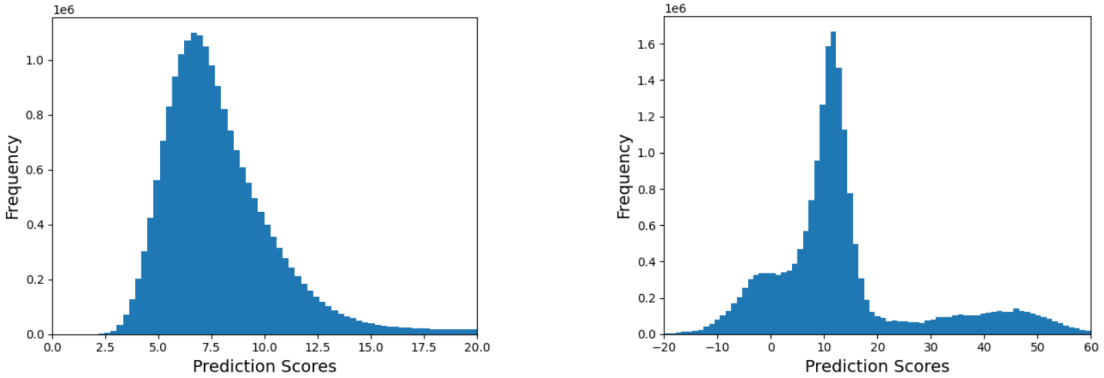
TABLE 5.2: Comparison between Ar-Flip1 and Ar-Flip2 attacks using the naive approach and Flip Attack using our approach at different change limits 0.4 and 1 on AraBERT using previously correctly classified offensive sentences (model's accuracy is 100% prior to the attack)

## 5.2 Hyperparameters of MarBERT and AraBERT attacks

The hyperparameters used to run the AraBERT and MarBERT attacks are shown in table 5.3. K denotes the top K predictions derived from the MLM output. The prediction score threshold is used to filter the K predictions from the MLM output based on their prediction score, which measures the model's confidence in this prediction. Due to the disparity in score ranges, we used different values for this hyperparameter in the two attacks. As indicated in figure 5.1, we created graphs for the prediction scores across the full correctly classified sentences using the two MLM models. Based on these graphs, we determined appropriate values to exclude outliers while allowing the bulk of good candidates to pass the filter.

The cosine similarity threshold is another filter for removing antonyms to the current word by removing any candidate word whose cosine similarity of its embedding vector with the current word vector is less than this threshold.

The perplexity threshold is a filter that prevents disjoined subwords from being passed as valid words during the subword candidate generation process. Due to memory constraints, $S_{\text{max-subwords}}$ represents the maximum amount of subwords that the current word is tokenized to that can go through the candidate generation process. $K_{\max}$ is the maximum number of candidates that can be considered for each subword in order to produce the entire permutation of all possible combinations for optimization reasons, as mentioned in the Methodology chapter.




(A) AraBERT MLM prediction scores graph

(B) MarBERT MLM prediction scores graph

FIGURE 5.1: Prediction Scores Graph

| Hyperparameter | AraBERT Attack | MarBERT Attack |
|---|---|---|
| K | 50 | 50 |
| Prediction Score Threshold | 4.5 | -10 |
| Cosine Similarity Threshold | 0.5 | 0.5 |
| Perplexity Threshold | 200 | 200 |
| $S_{\text{max-subwords}}$ | 8 | 8 |
| $K_{\text{max}}$ | 10 | 10 |

TABLE 5.3: Hyperparameters used in AraBERT and MarBERT Attacks

## 5.3 Study on AraBERT and MarBERT finetuned models using Offensive Data

We conducted a study utilizing only the Offensive sentences successfully classified by each model. We investigated the connection between changing the change limit and model accuracy. It goes without saying that the higher the limit, the more successful the attack. However, we believe that this can reduce the attack's perceptibility in particular types of attacks. In other circumstances, such as the Space Insertion, Special Characters Insertion, and Letters Separation attacks, we believe that the perceptibility is not as affected, even when performed on all of the words in the sentence.

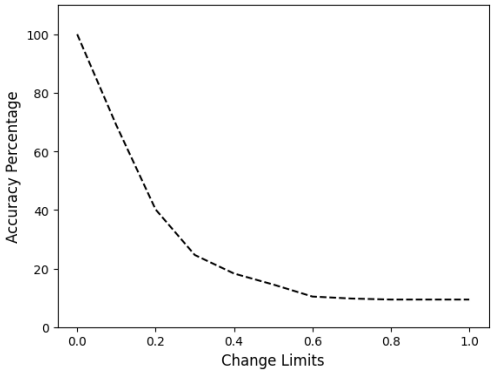
We can see in table 5.4 the effect of each attack on both models with change limit =1.

The relationships between the limit change and the AraBERT model's accuracy are illustrated in the graphs in figure 5.2 and 5.3 and the graphs of MarBERT model are shown in figure 5.4 and 5.5.

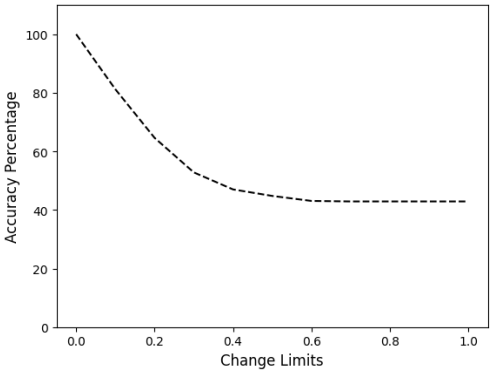
Separate Letters by Special Characters illustrates the perfect attack on the AraBERT model, with 0% model accuracy. However, we note that MarBERT training on social media posts gives it a significant advantage in terms of robustness against these kind of attacks. As nearly no attacks have been able to lower its accuracy below 75.6%. This will be covered in further depth in the following chapter, Results Analysis.

| Attack | AraBERT | MarBERT |
|---|---|---|
| MarBERT | 9.4% | 77.19% |
| AraBERT | 42.91% | 81.7% |
| Flip | 1.2% | 75.99% |
| English Flip | 1.2% | 77.06% |
| Middle Space | 21.88% | 81.56% |
| One Space Anywhere | 12.31% | 78.51% |
| Middle Special Character | 0.34% | 77.45% |
| One Special Character Anywhere | 0.51% | 76.66% |
| Separate Letters by Spaces | 24.62% | 84.75% |
| Separate Letters by Special Characters | **0%** | **75.6%** |
| Inner Shuffle | 17.44% | 79.58% |
| Adjective | 86.66% | 92.44% |

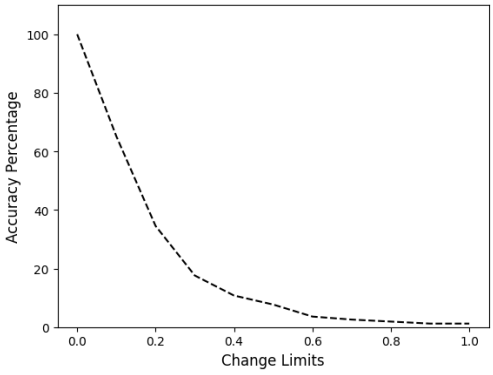
TABLE 5.4: Comparison between AraBERT & MarBERT Models accuracies after each attack on previously correctly classified offensive sentences (starting from 100% accuracy) with change limit = 1



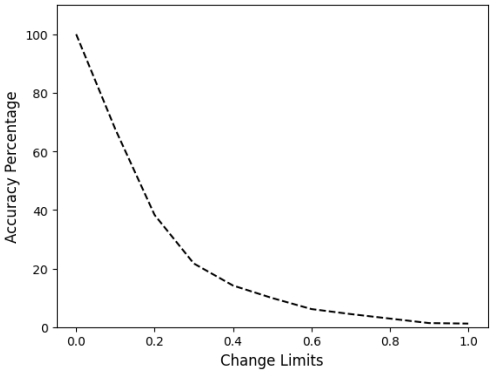

(A) MarBERT Attack



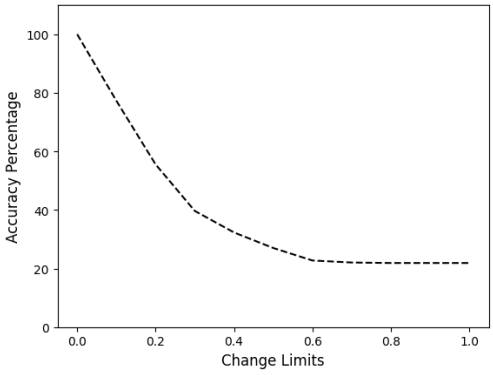

(B) AraBERT Attack



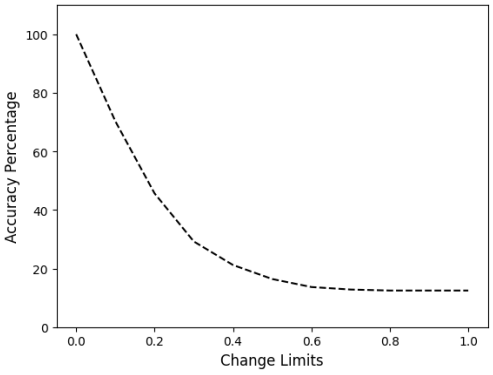

(C) Flip Attack




(D) English Flip Attack

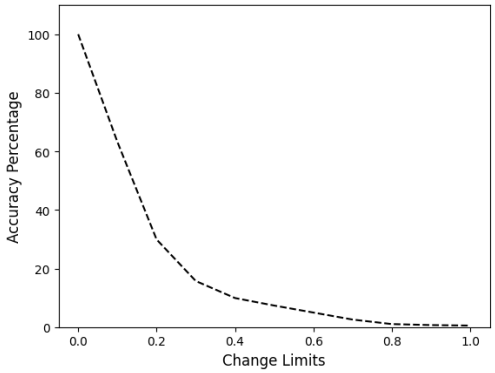



(E) Middle Space Attack

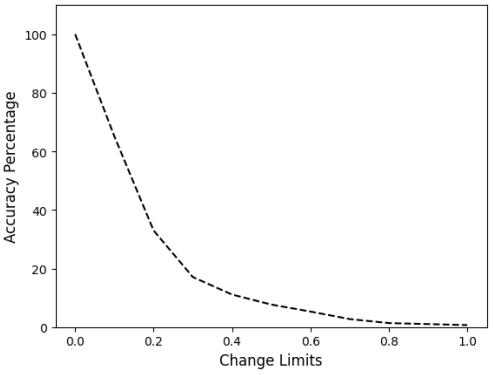



(F) One Space Anywhere Attack

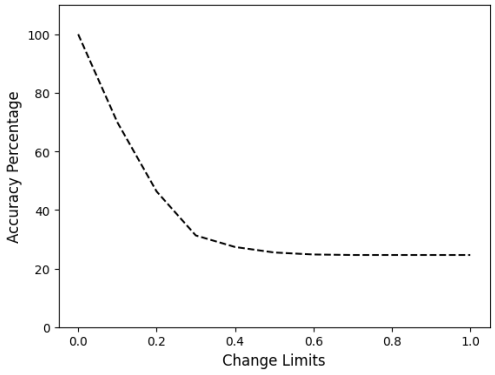
FIGURE 5.2: Plots of the accuracy change with the change limits in MarBERT, AraBERT, Flip, English Flip, Middle Space, and One Space Anywhere Attacks on AraBERT model



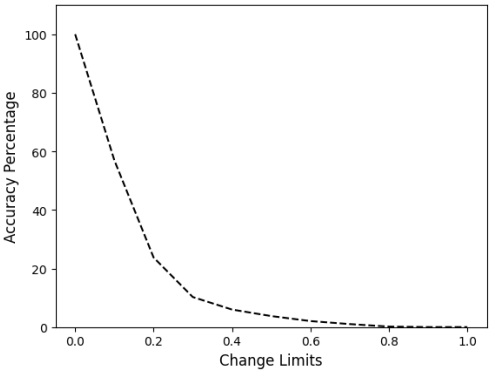

(A) Middle Special Character Attack



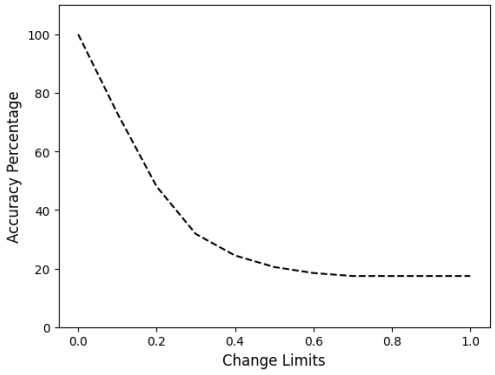

(B) One Special Character Attack



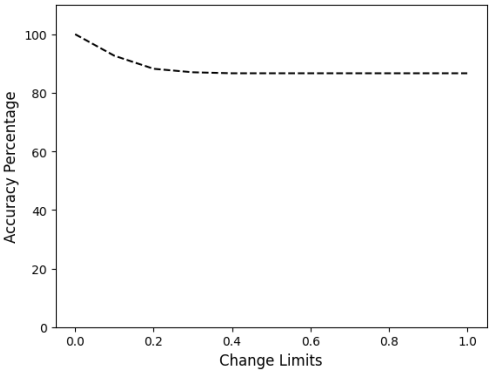

(C) Separate Letter by Spaces Attack




(D) Separate Letters by Special Characters Attack

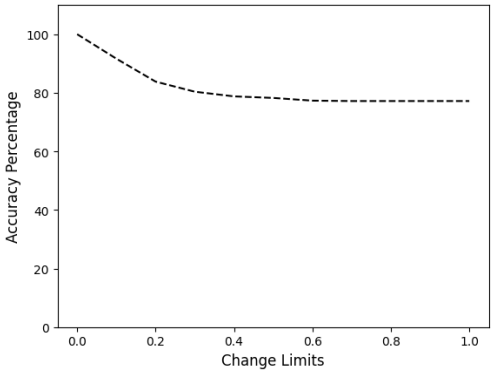



(E) Inner Shuffle Attack

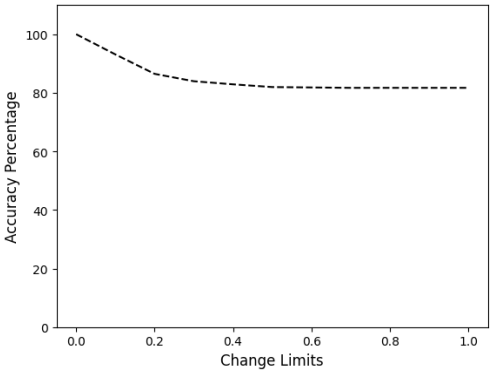



(F) Adjective Attack

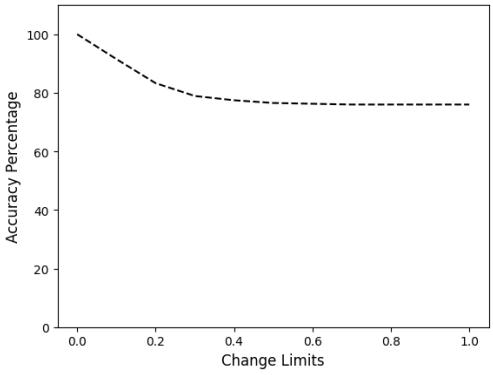
FIGURE 5.3: Plots of the accuracy change with the change limits in Middle Special Character, One Special Character Anywhere, Separate Letters by Spaces, Separate Letters by Special Characters, Inner Shuffle,and Adjective attacks on AraBERT model



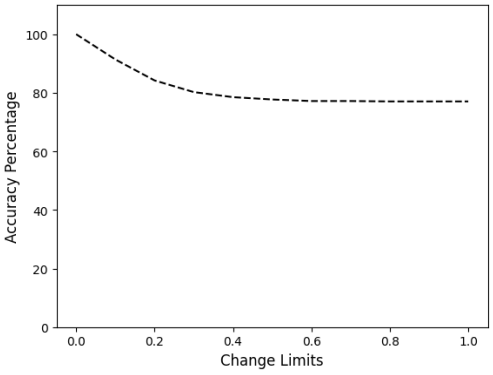

(A) MarBERT Attack



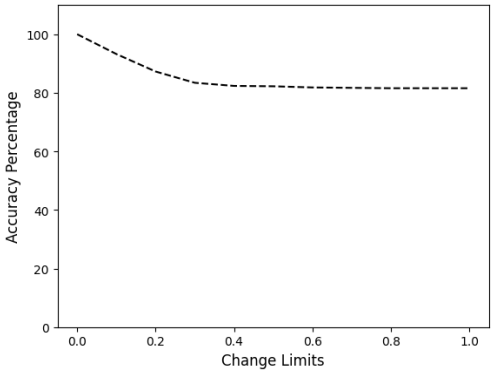

(B) AraBERT Attack



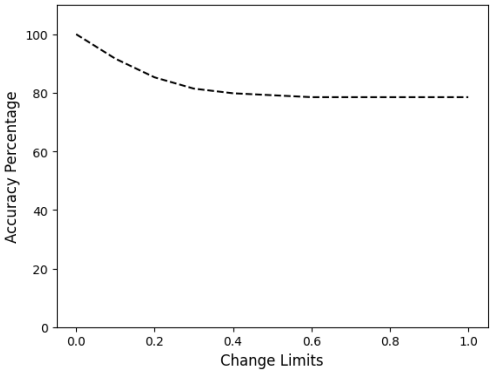

(C) Flip Attack




(D) English Flip Attack

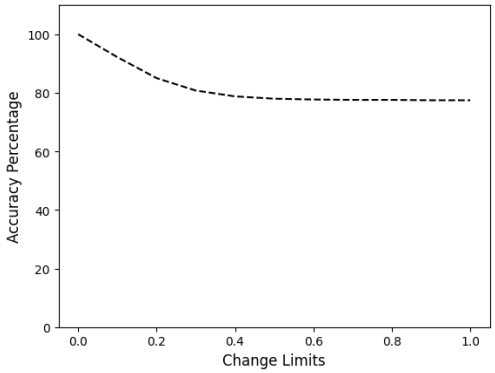



(E) Middle Space Attack

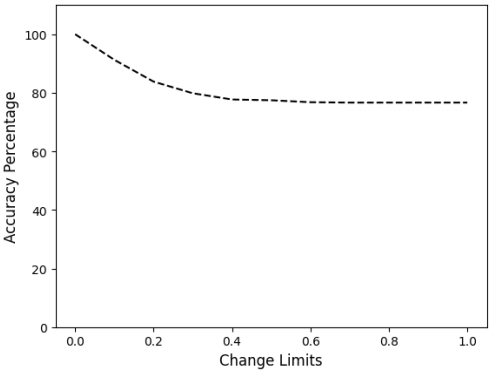



(F) One Space Anywhere Attack

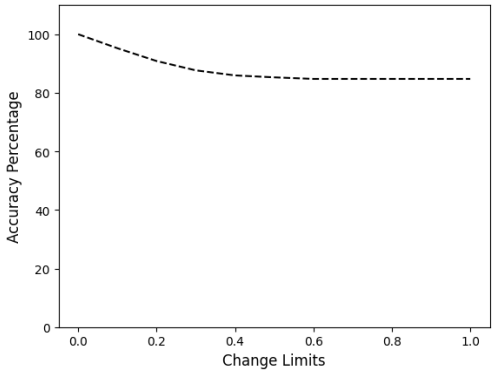
FIGURE 5.4: Plots of the accuracy change with the change limits in MarBERT, AraBERT, Flip, English Flip, Middle Space, and One Space Anywhere Attacks on MarBERT model



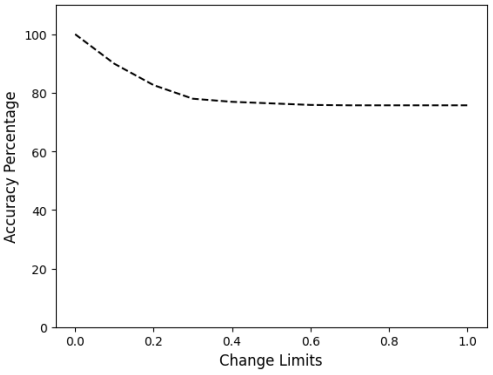

(A) Middle Special Character Attack



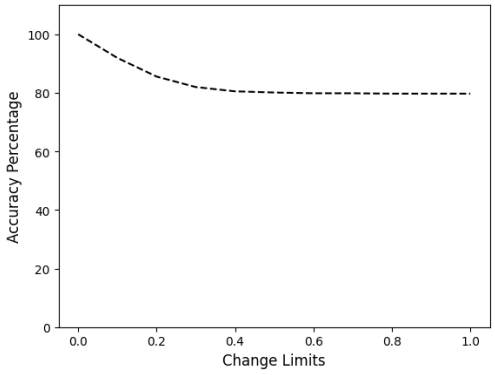

(B) One Special Character Attack



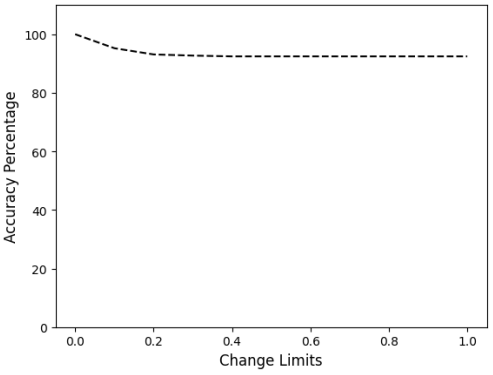

(C) Separate Letter by Spaces Attack




(D) Separate Letters by Special Characters Attack




(E) Inner Shuffle Attack




(F) Adjective Attack

FIGURE 5.5: Plots of the accuracy change with the change limits in Middle Special Character, One Special Character Anywhere, Separate Letters by Spaces, Separate Letters by Special Characters, Inner Shuffle,and Adjective attacks on MarBERT model

## 5.4 Study on AraBERT model using Offensive and Non-offensive Data

We conducted an experiment where we have run all of our 12 attacks on fine-tuned Arabert using the greedy search algorithm with a 40% change limit. In this experiment, we used all of the correctly classified OSACT5 test data to investigate the effect of the attacks on non-offensive sentences as well as offensive sentences. Although the purpose of our attack is to prevent the model from detecting an offensive sentence, we also wanted to test how the attacks affected benign sentences to see if the model produced any false positives. Further analysis on these results will be presented in the next chapter, Results Analysis.

Table 5.5 summarizes the effect of each attack on AraBERT model accuracy.

In addition, for each assault, we have compiled information on the average perturbation percentage, average query number, and average misclassified and failed sentence length for each attack in Table 5.7, Table 5.6 and Table 5.8 respectively.

| Attack | Model Accuracy | Offensive Accuracy | Non-offensive Accuracy |
|---|---|---|---|
| MarBERT | 39.07% | 17.60% | **47.21%** |
| AraBERT | 72.33% | 46.83% | 81.99% |
| Flip | **26.25%** | 9.74% | 32.51% |
| English Flip | 56.88% | 13.16% | 73.44% |
| Middle Space | 68.71% | 28.54% | 84.93% |
| One Space Anywhere | 61.24% | 19.65% | 77.00% |
| Middle Special Character | 57.30% | 9.05% | 75.58% |
| One Special Character Anywhere | 54.48% | 9.74% | 71.43% |
| Separate Letters by Spaces | 72.52% | 26.83% | 89.83% |
| Separate Letters by Special Characters | 58.43% | **5.47%** | 78.49% |
| Inner Shuffle | 51.47% | 22.90% | 62.30% |
| Adjective | 91.59% | 86.66% | 93.45% |

TABLE 5.5: AraBERT Model accuracy after each attack on previously correctly classified sentences (starting from 100% accuracy) with change limit = 0.4

| Attack | Entire Data | Misclassified Data | Data on which the Attack Failed |
|---|---|---|---|
| MarBERT | 83 | 83 | 82 |
| AraBERT | 39 | 40 | 39 |
| Flip | 74 | 76 | 66 |
| English Flip | 16 | 14 | 18 |
| Middle Space | 7 | 5 | **7** |
| One Space Anywhere | 18 | 14 | 20 |
| Middle Special Character | 113 | 89 | 131 |
| One Special Character Anywhere | 63 | 54 | 70 |
| Separate Letters by Spaces | **6** | **3** | 8 |
| Separate Letters by Special Characters | 114 | 84 | 135 |
| Inner Shuffle | 16 | 15 | 18 |
| Adjective | 13 | 25 | 12 |

Table 5.6: Average Query Numbers on AraBERT Model

| Attack | Entire Data | Misclassified Data | Data on Which Attack Failed |
|---|---|---|---|
| MarBERT | 23.18% | 18.12% | 31.08% |
| AraBERT | 23.02% | 18.42% | 24.77% |
| Flip | 20.83% | 18.14% | 28.39% |
| English Flip | 24.9% | 17.75% | 30.33% |
| Middle Space | 25.59% | 18.69% | 28.72% |
| One Space Anywhere | 26.17% | 17.61% | 31.58% |
| Middle Special Character | 25.45% | 16.32% | 32.25% |
| One Special Character Anywhere | 25.19% | 16.79% | 32.21% |
| Separate Letters by Spaces | 24.03% | 14.65% | 27.58% |
| Separate Letters by Special Characters | 25% | 14.9% | 32.19% |
| Inner Shuffle | 21.49% | 18.46% | 24.34% |
| Adjective | 6.4% | **11.91**% | 5.89% |

Table 5.7: Average Perturbation Percentage on AraBERT Model

| Attack | Misclassified Sentence | Failed Sentence | Misclassified Offensive Sentence | Misclassified Non-offensive Sentence |
|---|---|---|---|---|
| MarBERT | 15 | 8 | 14 | 17 |
| AraBERT | 17 | 10 | 17 | 17 |
| Flip | 14 | 7 | 17 | 13 |
| English Flip | 16 | 9 | 16 | 17 |
| Middle Space | 17 | 10 | 16 | 18 |
| One Space Anywhere | 17 | 10 | 16 | 17 |
| Middle Special Character | 17 | 9 | 17 | 17 |
| One Special Character Anywhere | 17 | 9 | 17 | 16 |
| Separate Letters by Spaces | 16 | 11 | 15 | 18 |
| Separate Letters by Special Characters | 17 | 9 | 17 | 17 |
| Inner Shuffle | 16 | 9 | 17 | 16 |
| Adjective | 16 | 12 | 15 | 17 |

Table 5.8: Average word length of the Misclassified, Failed, Misclassified Offensive, Misclassified Non-offensive sentences in the dataset in AraBERT experiment.
The average length of sentence in the entire data is 12 words.

# Chapter 6

# Results Analysis

In the analysis section, we delve into a detailed examination and interpretation of the results obtained in our study. Here, we critically evaluate the performance of the proposed methods, discuss the observed patterns, and provide insights into the implications of our findings. Through this analysis, we aim to gain a deeper understanding of the strengths and limitations of our approach, identify areas for improvement, and offer our opinions on the broader implications of our research.

## 6.1 Relationship between Attack Performance, Perceptibility, and Validity

As previously detailed, we conducted various experiments involving diverse attack methodologies. In this section, we present an analysis of their respective performance, drawing comparisons between the distinct attack types and evaluating the quality of the adversarial samples produced.

First, we discuss the word-level attacks where we generate word substitution candidates using a BERT-based model as an MLM for candidate prediction. We used AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020) as both candidate generators and target models, experimenting with all combinations. We realize that MarBERT, as a generator, in both targets outperforms AraBERT.

This observation hypothesizes that due to the quality of MarBERT's candidates generated, more candidates pass the rigorous filtration process than AraBERT,

which gives more space for finding a toxic candidate that will cause misclassification.

It also means that the wider the range of candidates produced, the more difficult for the target model to connect all the predictions to the same context, unlike humans, who can grasp that context effortlessly. Proving this, one specific result that stands out when MarBERT attacks AraBERT to reach a 9.4% accuracy, from a 100% accuracy, over the correctly classified offensive sentences as shown in table 5.4. This discussion also highlights the quality of the samples produced as they would be more perceptible to humans and more valid samples as the quality of the candidates are enhanced.

It is also clear that word-level attacks may have a much higher human perceptibility than character-level attacks if the candidates are correctly produced holding the same semantic meaning. However, due to the limitations of the model's understanding of any language and how not many words can semantically be connected precisely by their embeddings, this may affect the validity of the word-level attacks in contrast to the character-level attacks, which although are much less perceptible, they maintain the validity of the adversarial example and does not affect the meaning hidden in the context.

Another subtle adversarial technique at the word level is Adjective Attack, though perceptible, it demonstrates notably inferior efficacy relative to other methods. This stems from the architecture and training of the target models, as elaborated in the subsequent section. Specifically, the core semantic meaning of the word is retained, and owing to the tokenization methodology, the model is more prone to preserve the central token unaltered following perturbation, merely differing in an additional token that will not truly impact the prediction, unlike alteration of the focal token under attention. It also may affect the validity of the adversarial example in some cases. For example, a model for gender identification cannot alter the adjective's gender and have a valid sample.

Although character-level attacks are not as perceptible as word-level attacks, they are still perceptible to humans, as their design is directed towards delivering the exact meaning to the reader while totally deluding the model. As highlighted earlier, the validity is maintained throughout all character-level attacks, as the attacks target the syntax and not the semantics of the sample, and various measure are taken to ensure that.

The degree of perceptibility varies with the type of attack conducted. Although, the more perceptible a character-level attack is, the less likely it will affect the target model. Our intuition behind the character-level attacks is to break the tokens under strong attention from the model towards certain predictions so that these tokens are impalpable to the model, and hence the sample bypasses the detection. The more the token is indistinguishable from its origin, the more likely the model will fail to classify, but the less perceptible it is for humans as well.

Starting with the most perceptible attacks, Middle Space Attack and One Space Anywhere Attack divide the victim word by space which is barely noticeable by humans but can affect the model drastically as illustrated in table 5.4 where they degrade AraBERT to 21.88% and 12.31% respectively and MarBERT 81.56% and 78.51% in the same order.

The Middle Special Character Attack and One Special Character Anywhere Attack do exactly the same thing but use special characters instead of spaces. They actually perform much better especially on AraBERT as it reached 0.34% and 0.51% due to their respective effect. This results from the fact that tokenizers split tokens by spaces first so tokens are only split, while the special characters will be included in the split token so it will further conceal the original token and therefore affect the performance even further. It is worth mentioning that due to the huge number of combinations that could be produced in the second attack, a certain number of special characters is randomly picked to perform the splitting, this explains why the second attack may not always exceed the performance of the first.

Another two attacks that perform extremely well and rarely affect perceptibility are the Flip Attack and the English Flip Attack with an effect on AraBERT of 1.2% both and 75.99% and 77.06% on MarBERT respectively. These attacks split the token by flipping certain characters either into visually similar letters so that it won't be noticed by humans, or the phonetic mapping of the letter to the English language so it could be still recognizable but certainly confuses the model. They perform extremely well as the introdcution of the new letter in both cases may confuse the model even more than only a token split.

Inner Shuffle Attack although performs mildly average with respect to all other attacks introduced, it affects the perceptibility the most. Despite the fact that humans can read words correctly in that manner, it is still subjected to a margin

of error and also requires a certain amount of strain. Separate Letters by Spaces Attack was expected to perform better than other space attacks as it dissects the token. However, it did not, which may shed light on the ability of the model to recognize certain words that are consistently divided into letters.

We successfully degraded AraBERT's performance to a 0% for all the correctly detected offensive speeches using the Separate Letters by Special Characters Attack as tabel 5.4 shows. This attack was able to delude the model from detecting any single offensive speech inputted, which shows how Transformers-based models are extremely vulnerable to such attacks. This attack separates the token completely, however, it includes special characters that obscure the entire token and misguides the model with a totally different representation embedding. This attack also performed best out of all other attacks on MarBERT with an accuracy of 75%. This difference also reveals the robustness of MarBERT against these attacks with respect to AraBERT as we will explore in detail in the next section.

## 6.2 Transformer-based models: AraBERT and MarBERT

In our work, we used word-level and character-level attacks to target transformed-based models. Our intuition behind character-level attacks is based on targeting tokenization, a critical process on which the transformer relies. After the attack, flipping a character, separating letters with a space or special character, or shuffling letters can result in the previous token being broken down into numerous tokens. This can cause the model to be misled because the meaning of the individual token may be completely different and associated with a different context.

Because of the transformers' attention mechanism, classification in some tasks can be based on a few words in a sentence. In the hate speech detection task, for example, offensive sentence classification can be based on attending on the slur or offensive word in the phrase. If this word is split into many tokens, the model's decision may be affected significantly.

In our study, successful word-level attacks were based on Language Model (LM) generated candidates. We hypothesize that if the alternate term is not prevalent in this context in the training dataset of the target fine-tuned LM, LM can produce

valid alternatives to it that can mislead the target fine-tuned LM. Other word-level attacks that target adding a prefix or a suffix, such as the adjective attack, are less successful on transformers, and this is one example where the tokenization process can act as a defense.

In our study, we focused on two transformer-based models: AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020). We fine-tuned both models on the OSACT5 dataset (Mubarak et al., 2022) for offensive speech detection.

It is evident from Table 5.4 that MarBERT exhibits greater robustness compared to AraBERT. Additionally, in the case of MarBERT, the accuracy of figure 5.4 and 5.5 only experienced a minor decrease, with the most successful attack resulting in a decrease to 75.6% accuracy, as shown in subfigure 5.5d. Conversely, Arabert demonstrated higher vulnerability to attacks, as indicated by the success of multiple attacks in figure 5.2 and 5.3, leading to a significant decrease in accuracy, reaching 0%, as observed in subfigures 5.3d, 5.3a, 5.3b, 5.2c, and 5.2d .

MarBERT demonstrates greater robustness due to its pre-training on data from social media posts, specifically Arabic Twitter. Training on Arabic Twitter gives it an advantage to train on dialectal data enabling it to capture the specific characteristics of informal Arabic. This exposure to various vocabulary helps MarBERT against word-level attacks.

This type of data includes uncleaned text that may contain adversarial elements, such as words with inserted spaces or frequent use of punctuation like full stops and special characters. The utilization of uncleaned data during the training of MarBERT resembles the concept of adversarial training. This approach provides MarBERT with a significant advantage as it exposes the model to a wide range of linguistic and dialectal variations, along with their associated challenges. This exposure to varied linguistic patterns and challenges enhances MarBERT's overall performance and adaptability in real-world scenarios. As a result, MarBERT becomes more robust and better equipped to defend against character-level attacks.

In contrast, AraBERT is pre-trained on different MSA sources, all of which consist of cleaned and well-written text. While this ensures higher data quality, it also makes AraBERT more susceptible to attacks. The lack of exposure to uncleaned and adversarial data during training makes AraBERT less robust in handling such perturbations during inference.

Therefore, the contrasting training data sources contribute to the differing robustness levels of MarBERT and AraBERT, with MarBERT benefiting from adversarial training on uncleaned social media data and exhibiting greater resilience against attacks.

It is imperative to acknowledge that aside from disparities in their training data, AraBERT and MarBERT also employ different tokenization techniques. AraBERT utilizes the SentencePiece (Kudo and Richardson, 2018) tokenizer in unigram (Kudo, 2018a) mode with a vocabulary size of 64K, complemented by Farasa (Abdelali et al., 2016) preprocessing techniques. In contrast, MarBERT utilizes the WordPiece (Kudo, 2018b) tokenizer without the integration of the Farasa toolkit with a vocabulary size of 100K. This variation in tokenization strategies and vocabulary size could potentially account for MarBERT's observed greater resilience to adversarial attacks. The large vocabulary size can help MarBERT expand its knowledge of different dialectal nuances. While SentencePiece tokenizer in unigram mode along Farasa can efficiently handle the Arabic language's unique characteristics and effectively segment text into subword units, it can also make the model vulnerable to small changes.

## 6.3 Effects of Attacks on Offensive and Non-Offensive Sentences

Our analysis aims to compare the results and effects of adversarial attacks on both offensive and non-offensive sentences. Notably, we observed that the attacks had a significantly greater impact on sentences classified as offensive as shown in Table 5.5. This observation aligns with the primary objective of adversarial attacks, which is to manipulate the classification of offensive content.

The heightened effectiveness of attacks on offensive sentences can be attributed to several factors. Firstly, offensive sentences often exhibit a concentrated distribution of offensiveness, primarily residing in a smaller number of words. These words typically encompass explicit or derogatory language that triggers the offensive classification. By strategically modifying or replacing these targeted words, the attacks can significantly alter the classification of the entire sentence.

In contrast, non-offensive sentences lack a singular focal point for attack. Their classification relies on a more nuanced combination of linguistic features, context, and sentiment. As a result, the impact of adversarial attacks on non-offensive sentences tends to be relatively lower, and it is more challenging to disrupt the classification.

Furthermore, the nature of transformer-based and attention-based models plays a role in the varying effectiveness of attacks. These models assign higher attention weights to specific words based on their importance in determining the sentence's meaning. In offensive sentences, where the offensive intent is often expressed through specific words, the attention tends to be more concentrated. This increased attention makes it easier for adversarial attacks to manipulate the classification by targeting these crucial words. Conversely, in non-offensive sentences, the absence of a focal point and the distributed nature of linguistic features make it harder for attacks to achieve significant changes in the classification.

## 6.4 Advantages of our approach

As discussed in earlier chapters, we have implemented a different approach to creating adversarial attacks, which involves two crucial steps: candidate generation and search algorithm. By adopting this approach, we have witnessed a notable increase in the potential success of the attacks. Unlike conventional methods that are constrained by strict criteria and rely on probabilistic measures, our approach allows for the exploration of diverse combinations, thereby enabling us to precisely target the most vulnerable aspects of the sentence.

One significant advantage of our approach is its flexibility and generalizability. It can be easily extended to different types and levels of attacks, including character-level and word-level attacks. This versatility not only enhances the effectiveness of our attacks but also enables us to adapt our methodology to various scenarios.

To evaluate the efficacy of our approach, we conducted a comparative analysis with the naive approach. We performed this comparison in two distinct attack scenarios: the Adjective attack and the Flip attack. The results, which are presented in Tables 5.1 and 5.2, clearly demonstrate the significant improvement in attack performance achieved through our approach.

# Chapter 7

# Future Work

In this section, we aim to further explore and expand upon the findings and methodologies presented in this study. Building upon the knowledge gained from our current research, we present new directions and discuss additional aspects of adversarial machine learning in the Arabic language and NLP in general.

## 7.1 Explore Different Search Algorithms

Besides our work with greedy algorithm and PSO, other various search and optimization algorithms could also be leveraged to generate adversarial examples in natural language processing through optimal word substitution. Our future work may explore other potential approaches to be studied for enhancing the quality and robustness of adversarial examples as a solution to our combinatorial optimization problem, including:

1. **Simulated annealing (SA)** is a probabilistic optimization algorithm inspired by annealing in metallurgy (Kirkpatrick et al., 1983).

2. **Beam search** is a heuristic search algorithm that explores a subset of the search space by maintaining a beam of k candidate solutions at each step (Och and Ney, 2004).

3. **Ant colony optimization (ACO)** is a metaheuristic inspired by ant behavior (Dorigo et al., 2006). In ACO, "ants" traverse the search space and deposit pheromones indicating promising paths.

4. **Tabu search** is a metaheuristic using memory structures to avoid revisiting solutions (Glover, 1986).

5. **Differential evolution (DE)** combines genetic algorithms and numerical optimization (Storn and Price, 1997).

6. **Estimation of distribution algorithms (EDAs)** replace the variation operators of genetic algorithms with a statistical model that is updated over generations (Larrañaga and Lozano, 2001).

These search and optimization algorithms could potentially be used to enhance the generation of adversarial examples through word substitution. By adapting them to the problem and constraints of NLP, we may develop novel, robust techniques for crafting adversarial examples that overcome the limitations of existing methods.

## 7.2 Explore Alternative Candidate Generation

Various approaches can yield higher-quality word candidates, leading to improved adversarial examples that enhance the validity and perceptibility of the adversarial sample while further degrading the model's prediction accuracy. One promising candidate generation methodology, which warrants further exploration, is based on the concept of sememes. This approach has the potential to significantly advance the effectiveness of adversarial attacks while maintaining their imperceptibility to human observers

### 7.2.1 Sememes

In the field of linguistics, a sememe refers to the smallest unit of meaning in human languages. It represents the fundamental semantic unit from which the meaning of a word is constructed. In natural language processing (NLP), sememe knowledge bases have been developed to leverage the concept of sememes in practical applications. These knowledge bases, such as HowNet (Dong and Dong, 2003), annotate large numbers of English and Chinese words with predefined sets of sememes. HowNet, in particular, is a widely recognized sememe knowledge base that annotates over one hundred thousand words with approximately 2,000 sememes.

The sememe annotations in HowNet are assigned at the sense level, meaning that each sense of a word is annotated separately with its corresponding sememes.

The importance of sememe knowledge bases in Natural Language Processing (NLP) cannot be overstated, as they provide a structured and comprehensive representation of meaning for words and phrases. Sememe-based representations help to capture the semantic relationships between language elements, allowing NLP models to better understand the underlying structure and context in which words are used. As a result, sememe knowledge bases can significantly enhance the performance of NLP tasks, such as sentiment analysis, machine translation, and information extraction, by enabling a more accurate and granular understanding of language semantics.

#### 7.2.1.1 Sememe-based Word Substitution Method

The use of sememes in word substitution methods offers advantages over other approaches, such as word embedding-based, language model-based, and synonym-based methods. Sememe-based word substitution achieves a better balance between the quality and quantity of substitute words. Unlike word embedding and language model-based methods that may introduce inappropriate and low-quality substitutes, sememe-based methods avoid this issue by ensuring that substitute words share the same sememe annotations, maintaining semantic coherence and grammaticality in the adversarial examples. Additionally, sememe-based methods outperform synonym-based methods in terms of finding more substitute words, as HowNet's sememe annotations cover a wide range of word types.

For the Arabic language, which is characterized by its rich morphology, complex syntax, and diverse dialects, the development and integration of a sememe knowledge base can bring numerous benefits. Creating a sememe knowledge base tailored to Arabic can help address challenges arising from the language's unique properties, such as handling morphological variations, disambiguation of homonyms, and bridging the gap between Modern Standard Arabic and regional dialects. Furthermore, incorporating sememe-based representations into NLP models for Arabic can improve their ability to capture nuanced meanings and contexts, ultimately leading to more accurate and robust performance across a wide range of NLP applications in the Arabic-speaking world.

In addition, The advantages demonstrated by the sememe-based word substitution method suggest its potential effectiveness in the Arabic language. Considering the unique characteristics of Arabic, we believe that applying this method could yield promising results. To the best of our knowledge, sememe-based word substitution has not been extensively explored in the context of the Arabic language. Therefore, contributing by applying this method to Arabic opens up new possibilities for enhancing various adversarial attacks on natural language processing in Arabic and may lead to valuable insights and improvements in the field.

## 7.3 Adversarial Defences

Adversarial defenses are a crucial aspect of ensuring the security and robustness of machine learning models. Although most of the discussed attacks may seem intuitively trivial. There are no robust defences that can protect the model from such attacks. As these models become increasingly integrated into various applications in real life, they must be protected against malicious inputs, which are designed to exploit their vulnerabilities and cause them to produce incorrect predictions. We would like to explore various adversarial defences that encompass a range of techniques working together to safeguard machine learning systems from potential threats and maintain their reliability in an adversarial environment.

### 7.3.1 Adversarial Purification

**Diffusion Models for Adversarial Purification (Nie et al., 2022)** propose *DiffPure*, the first adversarial image purification method that leverages the forward and reverse processes of pre-trained diffusion models to remove adversarial perturbations while preserving label semantics. They provide a theoretical analysis quantifying the amount of noise required for purification and propose utilizing the adjoint method to defend against adaptive attacks. Extensive experiments show *DiffPure* achieves state-of-the-art results on adaptive attack benchmarks.

The authors make the following main contributions:

1. **DiffPure**: The first adversarial purification method using forward and reverse diffusion processes.

2. A theoretical analysis quantifying the required noise for effective purification without disrupting semantics.

3. The adjoint method enables defense against adaptive attacks. Experiments show state-of-the-art results on adaptive attack benchmarks.

4. The authors propose an innovative adversarial purification technique that provides strong defenses against modern adaptive attacks. Experiments demonstrate its state-of-the-art performance compared to previous methods. The theoretical analysis offers insights into how much noise is required for effective purification without disrupting semantics.

Future work could explore transferring the proposed *DiffPure* method to the natural language processing domain or exploring the concept of diffusion models for the denoising or purification of the adversarial samples.

The diffusion process shows promise for purifying adversarial text examples and defending against adaptive paraphrase attacks. Theoretical analysis can be expanded to quantify the amount of noise required to purify adversarial language without changing the original semantic meaning. The use of the adjoint method may enable defending against advanced adaptive paraphrase attacks that try to circumvent purification. Experiments on language benchmarks would demonstrate the effectiveness of the transferred purification technique.

Diffusion models could also be used as a generative model for the generation of even stronger adversarial attacks that can encourage more studies regarding the robustness of deep learning models.

Overall, this work could make promising contributions to the field of adversarial machine learning defenses that can be built upon for new applications. Transferring the image diffusion purification method to language is an important direction for future work.

### 7.3.2 Fuzzy Matching

Fuzzy matching, a technique widely used in various domains, refers to the process of finding approximate matches within a dataset. It employs algorithms that measure the similarity between strings, identifying partial matches when elements

are not exactly equal. Fuzzy matching has proven to be an invaluable tool in areas such as data cleaning, record linkage, and NLP.

Fuzzy matching algorithms typically rely on string similarity metrics, such as the Levenshtein distance, Jaro-Winkler distance, and cosine similarity, to determine the degree of similarity between two strings. These metrics capture the differences between strings by considering factors such as character insertions, deletions, and substitutions. In essence, fuzzy matching allows systems to recognize and process approximate matches, making it an adaptable and versatile technique for handling real-world data.

#### Fuzzy Matching for Adversarial Defense

Fuzzy matching could play a crucial role in enhancing the adversarial robustness of machine learning models. Below are potential applications of fuzzy matching in adversarial defense:

**Input Preprocessing** By using fuzzy matching techniques in input preprocessing, it may be possible to identify and mitigate the effects of adversarial perturbations before they reach the machine learning model. This could involve comparing the perturbed input against a database of known inputs, identifying the closest match, and replacing the perturbed input with the clean version.

**Adversarial Detection** Fuzzy matching may also contribute to the detection of adversarial examples. By comparing the similarity between input-output pairs in normal and adversarial conditions, a system can potentially differentiate between genuine input data and adversarial examples. If the similarity between input-output pairs significantly deviates from the norm, it could be indicative of an adversarial attack.

**Model Training and Augmentation** Incorporating fuzzy matching into the model training process could improve the model's resilience against adversarial attacks. By augmenting the training dataset with adversarial examples and their fuzzy matches, it may be possible to enhance the model's ability to recognize and

respond to adversarial perturbations, ultimately bolstering its robustness.

Fuzzy matching, with its ability to identify approximate matches and adapt to real-world data, holds promise for enhancing the adversarial robustness of machine learning models. Further research and development in this area may uncover new and innovative ways to leverage fuzzy matching techniques for adversarial defense, ensuring the security and reliability of future machine learning systems.

## 7.4 Adversarial Attacks

We also aim to explore and adapt adversarial attacks that have been developed for other languages and domains into the Arabic language. By extending the application of these attacks to Arabic, we can assess their effectiveness and compare their results against existing methods. This comparative analysis will provide valuable insights into the unique challenges and characteristics of the Arabic language in the context of adversarial attacks. Furthermore, by adapting and evaluating attacks from different languages, we can contribute to the development of robust and language-specific defense mechanisms in Arabic natural language processing applications.

In our exploration of adversarial attacks for the Arabic language, we specifically find the work done by (Gan et al., 2022) to hold potential. The paper presents a novel strategy for backdoor attacks in NLP models. Unlike traditional approaches that use triggers and altered labels, this strategy generates clean-labeled examples that can induce label changes when combined with the training set. By eliminating the need for external triggers, the proposed approach is both effective and difficult to defend against. Through extensive experiments, the paper demonstrates the success of this triggerless and clean-labeled attack strategy, marking a significant advancement in the field of NLP.

# Chapter 8

# Conclusion

In conclusion, this thesis has focused on adversarial attacks in NLP specifically focused on the Arabic language. Motivated by the concerning rise of hate speech and offensive content in online platforms, the research has highlighted the need for effective hate speech detection mechanisms and the challenges posed by anonymous online activities.

One major challenge addressed in this research pertains to adversarial attacks, where malicious users manipulate their content to bypass hate speech detection models. The study focused on 12 attacks, including both word-level and character-level techniques, and evaluated their impact on the AraBERT (Antoun et al., 2020) and MarBERT (Abdul-Mageed et al., 2020) models fine-tuned on the OSACT5 (Mubarak et al., 2022) hate speech detection dataset.

The primary objective of this study is to illustrate the impact of adversarial attacks on deep learning models and to draw attention to their vulnerabilities prior to their extensive deployment by the public. Additionally, we aim to emphasize that the showcased attacks serve as a means to explore and evaluate defenses against real-world scenarios, ultimately contributing to the establishment of a safer online environment. Notably, this research places a particular focus on the Arabic language, a domain that has received relatively limited attention in existing work pertaining to adversarial attacks in natural language processing (NLP).

# References


Ahmed Abdelali, Kareem Darwish, Nadir Durrani, and Hamdy Mubarak. Farasa: A fast and furious segmenter for arabic. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: Demonstrations*, pages 11–16, 2016.

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. Arbert & marbert: deep bidirectional transformers for arabic. *arXiv preprint arXiv:2101.01785*, 2020.

Naveed Akhtar and Ajmal Mian. Threat of adversarial attacks on deep learning in computer vision: A survey. *arXiv preprint arXiv:1801.00553*, 2018.

Jay Alammar. The illustrated transformer. *The Illustrated Transformer–Jay Alammar–Visualizing Machine Learning One Concept at a Time*, 27, 2018.

Basemah Alshemali and Jugal Kalita. Adversarial examples in arabic. In *2019 International Conference on Computational Science and Computational Intelligence (CSCI)*, pages 371–376. IEEE, 2019.

Basemah Alshemali and Jugal Kalita. Character-level adversarial examples in arabic. pages 9–14, 2021.

Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bo-Jhang Ho, Mani Srivastava, and Kai-Wei Chang. Generating natural language adversarial examples. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2890–2896, Brussels, Belgium, October-November 2018a. Association for Computational Linguistics. doi: 10.18653/v1/D18-1316. URL https://aclanthology.org/D18-1316.

Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bok-Jin Ho, Mani Srivastava, and Kai-Wei Chang. Generating natural language adversarial examples. In

*Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2890–2896, 2018b.

Wissam Antoun, Fady Baly, and Hazem Hajj. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France, May 2020. European Language Resource Association. ISBN 979-10-95546-51-1. URL https://aclanthology.org/2020.osact-1.2.

Anish Athalye, Logan Engstrom, Andrew Ilyas, and Kevin Kwok. Synthesizing robust adversarial examples. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. doi: 10.1109/TPAMI.2013.50.

Adam L. Berger, Stephen A. Della Pietra, and Vincent J. Della Pietra. A maximum entropy approach to natural language processing. *Computational Linguistics*, 22(1):39–71, 1996. URL https://aclanthology.org/J96-1002.

Arjun Nitin Bhagoji, Daniel Cullina, Chawin Sitawarin, and Prateek Mittal. Enhancing robustness of machine learning systems via data transformations. In *2018 52nd Annual Conference on Information Sciences and Systems (CISS)*, pages 1–5, 2018. doi: 10.1109/CISS.2018.8362326.

Daniel M. Bikel, Scott Miller, Richard Schwartz, and Ralph Weischedel. Nymble: a high-performance learning name-finder. In *Fifth Conference on Applied Natural Language Processing*, pages 194–201, Washington, DC, USA, March 1997. Association for Computational Linguistics. doi: 10.3115/974557.974586. URL https://aclanthology.org/A97-1029.

T.L. Booth and R.A. Thompson. Applying probability measures to abstract languages. *IEEE Transactions on Computers*, C-22(5):442–450, 1973. doi: 10.1109/T-C.1973.223746.

L. Cagnina, S. Esquivel, and C. Coello Coello. Application of particle swarm optimization to pattern classification. *Journal of Artificial Evolution and Applications*, 2008:746158, 2008.

Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 39–57, 2017.

Nicholas Carlini and David A Wagner. Audio adversarial examples: Targeted attacks on speech-to-text. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6295–6304, 2018.

Eugene Charniak. Statistical parsing with a context-free grammar and word statistics. In *AAAI/IAAI*, 1997.

Eugene Charniak. A maximum-entropy-inspired parser. In *1st Meeting of the North American Chapter of the Association for Computational Linguistics*, 2000. URL https://aclanthology.org/A00-2018.

Eugene Charniak, Curtis Hendrickson, Neil Jacobson, and Mike Perkowitz. Equations for part-of-speech tagging. In *AAAI*, volume 11, pages 784–789. Citeseer, 1993.

Peng Chen, Yingqi Lai, Han Liu, Teng Sun, and Kangjie Su. Texttrojan: A practical trojaning attack against nlp models. In *Proceedings of the 29th USENIX Security Symposium*, pages 2693–2708, 2020.

Pin-Yu Chen, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, pages 15–26, 2017.

Ming Cheng, Jinfeng Le, Pin-Yu Chen, and Cho-Jui Hsieh. Seq2sick: Evaluating the robustness of sequence-to-sequence models with adversarial examples. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8034–8041, 2020.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation.

In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1179. URL https://aclanthology.org/D14-1179.

Noam Chomsky. *Syntactic Structures*. Mouton and Co., The Hague, 1957.

Shammur Absar Chowdhury, Hamdy Mubarak, Ahmed Abdelali, Soon-gyo Jung, Bernard J. Jansen, and Joni Salminen. A multi-platform Arabic news comment dataset for offensive language detection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 6203–6212, Marseille, France, May 2020. European Language Resources Association. ISBN 979-10-95546-34-4. URL https://aclanthology.org/2020.lrec-1.761.

Kenneth Ward Church. A stochastic parts program and noun phrase parser for unrestricted text. In *Second Conference on Applied Natural Language Processing*, pages 136–143, Austin, Texas, USA, February 1988. Association for Computational Linguistics. doi: 10.3115/974235.974260. URL https://aclanthology.org/A88-1019.

Jeremy Cohen, Amir Rosenfeld, and J Zico Kolter. Certified adversarial robustness via randomized smoothing. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 1310–1320, 2019.

Michael Collins. Three generative, lexicalised models for statistical parsing. In *35th Annual Meeting of the Association for Computational Linguistics and 8th Conference of the European Chapter of the Association for Computational Linguistics*, pages 16–23, Madrid, Spain, July 1997. Association for Computational Linguistics. doi: 10.3115/976909.979620. URL https://aclanthology.org/P97-1003.

Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, Sep 1995. ISSN 1573-0565. doi: 10.1007/BF00994018. URL https://doi.org/10.1007/BF00994018.

Marta R Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, 2022.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423.

Zhendong Dong and Qiang Dong. Hownet - a hybrid language and knowledge resource. In *International Conference on Natural Language Processing and Knowledge Engineering, 2003. Proceedings. 2003*, pages 820–824, 2003. doi: 10.1109/NLPKE.2003.1276017.

Marco Dorigo, Mauro Birattari, and Thomas Stutzle. Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1(4):28–39, 2006. doi: 10.1109/MCI.2006.329691.

Russell Eberhart and James Kennedy. A new optimizer using particle swarm theory. In *Proceedings of the Sixth International Symposium on Micro Machine and Human Science*, pages 39–43, 1995.

Javid Ebrahimi, Vinay Rao, Daniel Lowd, and Dejing Dou. Hotflip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 31–36, 2017.

Javid Ebrahimi, Aditi Rao, Daniel Lowd, and Dejing Dou. Hotflip: White-box adversarial examples for text classification. In *Proceedings of the Workshop on Adversarial Training*, pages 31–36, 2018.

Steffen Eger and Yannik Benz. From hero to z\'eroe: A benchmark of low-level adversarial attacks. *arXiv preprint arXiv:2010.05648*, 2020.

Ibrahim Abu El-Khair. 1.5 billion words arabic corpus. *arXiv preprint arXiv:1611.04033*, 2016.

Ashraf Elnagar and Omar Einea. Brad 1.0: Book reviews in arabic dataset. In *2016 IEEE/ACS 13th International Conference of Computer Systems and Applications (AICCSA)*, pages 1–8. IEEE, 2016.

Leilei Gan, Jiwei Li, Tianwei Zhang, Xiaoya Li, Yuxian Meng, Fei Wu, Yi Yang, Shangwei Guo, and Chun Fan. Triggerless backdoor attack for NLP tasks with clean labels. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2942–2952, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.naacl-main.214. URL https://aclanthology.org/2022.naacl-main.214.

Ji Gao, Jack Lanchantin, Mary Lou Soffa, and Yanjun Qi. Black-box generation of adversarial text sequences to evade deep learning classifiers. In *2018 IEEE Security and Privacy Workshops (SPW)*, pages 50–56. IEEE, 2018a.

Jia Gao, Jack Lanchantin, Mary Lou Soffa, and Yanjun Qi. Black-box generation of adversarial text sequences to evade deep learning classifiers. In *2018 IEEE Security and Privacy Workshops (SPW)*, pages 50–56, 2018b.

Fred Glover. Future paths for integer programming and links to artificial intelligence. *Computers & operations research*, 13(5):533–549, 1986.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014a.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014b.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

Kathrin Grosse, Praveen Manoharan, Nicolas Papernot, Michael Backes, and Patrick McDaniel. On the (statistical) detection of adversarial examples. *arXiv preprint arXiv:1702.06280*, 2017.

R. Hassan, B. Cohanim, O. De Weck, and G. Venter. A comparison of particle swarm optimization and the genetic algorithm. In *Proceedings of the 46th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, pages 18–21, 2005.

Dan Hendrycks, Kevin Zhao, Rahman Basartan, Jacob Steinhardt, and Dawn Song. Natural adversarial examples. *arXiv preprint arXiv:2102.11295*, 2021.

Nobuaki Higashi and Hitoshi Iba. Particle swarm optimization with gaussian mutation. In *Proceedings of the 2003 IEEE Swarm Intelligence Symposium*, pages 72–79. IEEE, 2003.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Zhiting Hu, Zichao Yang, Xiaodan Liang, Ruslan Salakhutdinov, and Eric P Xing. Toward controlled generation of text. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 2091–2100, 2019.

F. Jelinek. The development of an experimental discrete dictation recognizer. *Proceedings of the IEEE*, 73(11):1616–1624, 1985. doi: 10.1109/PROC.1985.13343.

Robin Jia and Percy Liang. Adversarial examples for evaluating reading comprehension systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2021–2031, 2017.

Robin Jia, Ahmed Salem, and Percy Liang. Adversarialqa: A large-scale dataset for adversarial question answering. *arXiv preprint arXiv:2005.14165*, 2020.

Di Jin, Zhijing Jin, Joey Zhou, and Peter Szolovits. Is BERT really robust? a strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8018–8025, 2020.

Nanda Kambhatla. Combining lexical, syntactic, and semantic features with maximum entropy models for information extraction. In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, pages 178–181, Barcelona, Spain, July 2004. Association for Computational Linguistics. URL https://aclanthology.org/P04-3022.

James Kennedy and Russell C Eberhart. A discrete binary version of the particle swarm algorithm. In *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*, pages 4104–4108. IEEE, 1997.

Yoon Kim. Convolutional neural networks for sentence classification. *arXiv preprint arXiv:1408.5882*, 2014.

Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. *arXiv preprint arXiv:1804.10959*, 2018a.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. *arXiv preprint arXiv:1804.10959*, 2018b.

Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226*, 2018.

Pedro Larrañaga and Jose A Lozano. *Estimation of distribution algorithms: A new tool for evolutionary computation*, volume 2. Springer Science & Business Media, 2001.

Linyang Li, Ruotian Ma, Qipeng Guo, Xiangyang Xue, and Xipeng Qiu. Bert-attack: Adversarial attack against bert using bert. *arXiv preprint arXiv:2004.09984*, 2020.

Xiaoxue Li, Yanpeng Zhang, Bo Chen, and Chenggang Ju. Generating adversarial examples for text classification with word and character-level perturbations. *Information Sciences*, 559:203–219, 2021.

Xiaodong Liu, Pengcheng He, Weizhu Chen, and Jianfeng Gao. Multi-task deep neural networks for natural language understanding. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4487–4496, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1441. URL https://aclanthology.org/P19-1441.

Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. The MIT Press, Cambridge, Massachusetts, 1999. URL http://nlp.stanford.edu/fsnlp/.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf.

Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Proc. Interspeech 2010*, pages 1045–1048, 2010. doi: 10.21437/Interspeech.2010-343.

Takeru Miyato, Andrew M Dai, and Ian Goodfellow. Adversarial training methods for semi-supervised text classification. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. Deepfool: a simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2574–2582, 2016.

Hamdy Mubarak, Sabit Hassan, and Shammur Absar Chowdhury. Emojis as anchors to detect arabic offensive language and hate speech. *CoRR*, abs/2201.06723, 2022. URL https://arxiv.org/abs/2201.06723.

Weili Nie, Brandon Guo, Yujia Huang, Chaowei Xiao, Arash Vahdat, and Anima Anandkumar. Diffusion models for adversarial purification. *arXiv preprint arXiv:2205.07460*, 2022.

Ossama Obeid, Nasser Zalmout, Salam Khalifa, Dima Taji, Mai Oudah, Bashar Alhafni, Go Inoue, Fadhl Eryani, Alexander Erdmann, and Nizar Habash. CAMeL tools: An open source python toolkit for Arabic natural language processing. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 7022–7032, Marseille, France, May 2020a. European Language Resources Association. ISBN 979-10-95546-34-4. URL https://aclanthology.org/2020.lrec-1.868.

Ossama Obeid, Nasser Zalmout, Salam Khalifa, Dima Taji, Mai Oudah, Bashar Alhafni, Go Inoue, Fadhl Eryani, Alexander Erdmann, and Nizar Habash.

Camel tools: An open source python toolkit for arabic natural language processing. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 7022–7032, 2020b.

Franz Josef Och and Hermann Ney. The alignment template approach to statistical machine translation. *Computational Linguistics*, 30(4):417–449, 2004. doi: 10.1162/0891201042544884. URL https://aclanthology.org/J04-4002.

M. G. H. Omran, A. P. Engelbrecht, and A. Salman. Particle swarm optimization method for image clustering. *International Journal of Pattern Recognition and Artificial Intelligence*, 19(03):297–321, 2005.

OSACT5. The 5th workshop on open-source arabic corpora and processing tools, arabic hate speech 2022 shared task. https://sites.google.com/view/arabichate2022/home?authuser=0, 2022.

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP 2002)*, pages 79–86. Association for Computational Linguistics, July 2002. doi: 10.3115/1118693.1118704. URL https://aclanthology.org/W02-1011.

Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, pages 506–519, 2016a.

Nicolas Papernot, Patrick McDaniel, Xi Wu, Somesh Jha, and Ananthram Swami. Distillation as a defense to adversarial perturbations against deep neural networks. In *Proceedings of the 2016 IEEE Symposium on Security and Privacy (SP)*, pages 582–597. IEEE, 2016b.

Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pages 506–519, 2017.

Jeffrey Pennington, Richard Socher, and Christopher Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543,

Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1162. URL https://aclanthology.org/D14-1162.

Fernando C.N. Pereira and David H.D. Warren. Definite clause grammars for language analysis—a survey of the formalism and a comparison with augmented transition networks. *Artificial Intelligence*, 13(3):231–278, 1980. ISSN 0004-3702. doi: https://doi.org/10.1016/0004-3702(80)90003-X. URL https://www.sciencedirect.com/science/article/pii/000437028090003X.

M. Ross Quillian. Semantic networks. In Marvin L. Minsky, editor, *Semantic Information Processing*. MIT Press, 1968.

J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, Mar 1986. ISSN 1573-0565. doi: 10.1007/BF00116251. URL https://doi.org/10.1007/BF00116251.

L.R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Lance Ramshaw and Mitch Marcus. Text chunking using transformation-based learning. In *Third Workshop on Very Large Corpora*, 1995. URL https://aclanthology.org/W95-0107.

Sebastian Ruder, Matthew E. Peters, Swabha Swayamdipta, and Thomas Wolf. Transfer learning in natural language processing. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Tutorials*, pages 15–18, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-5004. URL https://aclanthology.org/N19-5004.

Alexander M. Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 379–389, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1044. URL https://aclanthology.org/D15-1044.

Pouya Samangouei, Maya Kabkab, and Rama Chellappa. Defense-gan: Protecting classifiers against adversarial attacks using generative models. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1778–1787, 2018.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

Yash Sharma and Pin-Yu Chen. Breaking the madry defense model with l˙1-based adversarial examples. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 1624–1633, 2018.

Yuhui Shi and Russell C Eberhart. A modified particle swarm optimizer. In *Proceedings of the IEEE International Conference on Evolutionary Computation*, pages 69–73. IEEE, 1998.

S. Silva, R. C. Prati, and F. J. Von Zuben. Part-of-speech tagging with evolutionary techniques. *Applied Soft Computing*, 12(4):1244–1256, 2012.

Jacob Steinhardt, Pang Wei Koh, and Percy S Liang. Certified defenses for data poisoning attacks. In *Advances in Neural Information Processing Systems*, volume 30, pages 3520–3532, 2017.

Rainer Storn and Kenneth Price. Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11:341–359, 1997.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/file/a14ac55a4f27472c5d894ec1c3c743d2-Paper.pdf.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.

Mohamed A. Taher. Arabic stop words. https://github.com/mohataher/arabic-stop-words/blob/master/list.txt, 2023.

Florian Tramèr, Alexey Kurakin, Nicolas Papernot, Ian Goodfellow, Dan Boneh, and Patrick McDaniel. Ensemble adversarial training: Attacks and defenses. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, ukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Eric Wallace, Allyson Mitchell, and Matt Gardner. Universal adversarial triggers for attacking and analyzing nlp. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2153–2162, 2019.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5446. URL https://aclanthology.org/W18-5446.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/4496bf24afe7fab6f046bf4923da8de6-Paper.pdf.

Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 9154–9160, 2020.

Joseph Weizenbaum. Eliza—a computer program for the study of natural language communication between man and machine. *Commun. ACM*, 9(1):36–45, jan 1966. ISSN 0001-0782. doi: 10.1145/365153.365168. URL https://doi.org/10.1145/365153.365168.

Terry Winograd. Understanding natural language. *Cognitive Psychology*, 3(1): 1–191, 1972. ISSN 0010-0285. doi: https://doi.org/10.1016/0010-0285(72)90002-3. URL https://www.sciencedirect.com/science/article/pii/0010028572900023.

W. A. Woods. Progress in natural language understanding: An application to lunar geology. In *Proceedings of the June 4-8, 1973, National Computer Conference and Exposition*, AFIPS '73, page 441–450, New York, NY, USA, 1973. Association for Computing Machinery. ISBN 9781450379168. doi: 10.1145/1499586.1499695. URL https://doi.org/10.1145/1499586.1499695.

Cihang Xie, Jianyu Wang, Zhishuai Zhang, Zhou Ren, and Alan Yuille. Mitigating adversarial effects through randomization. In *Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2780–2789. IEEE, 2017.

Han Xu, Yunbin Ma, Haochen Liu, Debayan Deb, Hui Liu, Jiliang Tang, and Anil K Jain. Adversarial attacks and defenses in images, graphs and text: A review. *Information Fusion*, 58:52–77, 2019.

Weilin Xu, David Evans, and Yanjun Qi. Feature squeezing: Detecting adversarial examples in deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1681–1690, 2017.

Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/dc6a7e655d7e5840e66733e9ee67cc69-Paper.pdf.

Marcos Zampieri, Preslav Nakov, Sara Rosenthal, Pepa Atanasova, Georgi Karadzhov, Hamdy Mubarak, Leon Derczynski, Zeses Pitenis, and Çağrı Çöltekin. SemEval-2020 task 12: Multilingual offensive language identification in social media (OffensEval 2020). In *Proceedings of the Fourteenth Workshop on Semantic Evaluation*, pages 1425–1447, Barcelona (online), December 2020. International Committee for Computational Linguistics. doi: 10.18653/v1/2020.semeval-1.188. URL https://aclanthology.org/2020.semeval-1.188.

Yuan Zang, Fanchao Qi, Chenghao Yang, Zhiyuan Liu, Meng Zhang, Qun Liu, and Maosong Sun. Word-level textual adversarial attacking as combinatorial optimization. *arXiv preprint arXiv:1910.12196*, 2019.

Imad Zeroual, Dirk Goldhahn, Thomas Eckart, and Abdelhak Lakhouaja. Osian: Open source international arabic news corpus-preparation and integration into the clarin-infrastructure. In *Proceedings of the fourth arabic natural language processing workshop*, pages 175–182, 2019.

Yuan Zhang, Jason Baldridge, and Luheng He. Paws: Paraphrase adversaries from word scrambling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1, pages 1298–1308, 2019.

Zhengliang Zhao, Dheeru Dua, and Sameer Singh. Generating natural adversarial examples. In *Proceedings of the 2018 International Conference on Learning Representations (ICLR)*, 2018.